



Propeller Heads Turbine

Security Assessment

March 31, 2026

Prepared for:

Markus Schmitt

Propeller Heads

Prepared by: **Samuel Moelius, Scott Arciszewski, Paul Bottinelli, and Omar Inuwa**

Table of Contents

Table of Contents	1
Project Summary	3
Executive Summary	4
Project Goals	7
Project Targets	8
Project Coverage	9
Automated Testing	12
Codebase Maturity Evaluation	13
Summary of Findings	15
Detailed Findings	17
1. Documentation for running tests needs to be updated	17
2. serialize_struct argument does not match number of serialize_field calls	19
3. PermitSingle and PermitBatch's From implementations can panic in U48::from	21
4. u256_to_naive_datetime can produce values before UNIX epoch	24
5. Invalid bit-length comparison in reduce_precision	25
6. Recursive price filtering allows oracle threshold bypass	27
7. SIWE message version field is not validated	29
8. U256 arithmetic errors silenced with unwrap_or	31
9. Authentication credentials leak when transmitted over unencrypted HTTP	33
10. Potential integer overflow in function scale_up_input	35
11. ECDSA signature malleability	37
12. Missing validation of user balance vector length	39
13. Missing block number validation in pool state updates	41
14. Race conditions are present in filesystem operations	42
15. Unable to safely change hook address	44
16. Unbounded intent array enables denial of service of settlement system	46
17. Nonstandard ERC-20 tokens cause swap failures due to unsafe transfer checks	49
18. Insufficient pre-execution validation enables single-user actions to cause batch settlement failures	51
19. Remove liquidity operations lack slippage protection, enabling unfavorable token ratios	53
20. Off-by-one-second discrepancy in speedbump enforcement between on-chain and off-chain paths	55



21. Missing minimum buy amount validation allows users to receive less than specified minimum	57
A. Vulnerability Categories	59
B. Code Maturity Categories	61
C. Known Issues and Planned Improvements	63
Rust Code	63
Price Providers	64
Solidity Code	64
Relevant to turbine-bots Repository	64
D. Non-Security-Related Recommendations	65
E. Recommendations for Addressing Recoverable Errors	73
Panicking in From implementations	73
Replacing Infallible APIs with Fallible APIs	73
Examples of Where Panicking Code Is Currently Used	75
F. Coverage Analysis	77
Solidity Code	77
Rust Code	78
G. Rust Functions That Were Fuzzed	81
H. Recommendations for Adding a SortedTokenPair Type	91
I. Fix Review Results	93
Detailed Fix Review Results	94
J. Fix Review Status Categories	99
K. Review of Known Issues and Planned Improvements PRs	100
About Trail of Bits	104
Notices and Remarks	105



Project Summary

Contact Information

The following project manager was associated with this project:

Tara Goodwin-Ruffus, Project Manager
tara.goodwin-ruffus@trailofbits.com

The following engineering director was associated with this project:

Benjamin Samuels, Engineering Director, Blockchain
benjamin.samuels@trailofbits.com

The following consultants were associated with this project:

Samuel Moelius, Consultant **Scott Arciszewski**, Consultant
samuel.moelius@trailofbits.com scott.arciszewski@trailofbits.com

Paul Bottinelli, Consultant **Omar Inuwa**, Consultant
paul.bottinelli@trailofbits.com omar.inuwa@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
November 10, 2025	Pre-project kickoff call
November 24, 2025	Status update meeting #1
December 8, 2025	Status update meeting #2
December 15, 2025	Delivery of report draft
December 15, 2025	Report readout meeting
December 16, 2025	Delivery of updates to TOB-TURBINE-21 based on client feedback
December 17, 2025	Delivery of updates to TOB-TURBINE-6 and TOB-TURBINE-15 based on client feedback
January 2, 2026	Delivery of final comprehensive report
March 17, 2026	Completion of fix review
March 31, 2026	Delivery of updated report with fix review appendix



Executive Summary

Engagement Overview

Propeller Heads engaged Trail of Bits to review the security of the Turbine “trustless dark solver.”

Turbine is an exchange for cryptocurrencies. Most computations happen off-chain, and only settlement transactions are submitted on-chain. Turbine runs in a trusted execution environment (TEE). User orders are confidential and are not revealed until they are settled on-chain.

A team of four consultants conducted the review from November 17 to December 11, 2025, for a total of eight engineer-weeks of effort. Our testing efforts focused on logic errors that could require the protocol to be redesigned. With full access to source code and documentation, we performed static and dynamic testing of the codebase, using automated and manual processes.

Observations and Impact

At the start of the audit, we were provided a list of 16 known issues and planned improvements ([appendix C](#)). We applaud Propeller Heads for seeking a review before the product is finished. However, some of the known issues will require considerable effort to resolve. For example, replacing uses of `panic`, `unwrap`, and `expect` for recoverable errors will require API changes. Changing the API of a function could require changes to the APIs of its callers. Recommendations for addressing such changes are provided in [appendix E](#).

We verified that Propeller Heads deployed Turbine to [Phala Cloud](#), a TEE provider. However, the deployed version did not measure the Speedex binary that Turbine relies on for matching buy and sell orders. Thus, it is not currently possible to get a complete measurement of the TEE code that one would interact with. Propeller Heads is working on a Rust port of the Speedex binary, which will facilitate attestation from Phala Cloud.

We ran both the Solidity and Rust tests, and all succeeded. However, Propeller Heads had to provide us specific values of two environment variables for the tests to work (see [TOB-TURBINE-1](#)). We recommend that this situation be resolved so that arbitrary users can run the tests without needing to request values for those environment variables.

We also computed the project’s test coverage for both types of tests, neither of which were 100%. We generally recommend 100% test coverage for Solidity code since it has the most direct control over user funds. The Rust code is not blockchain code, but it does have influence over user funds. Thus, we recommend achieving as close to 100% as possible for Rust test coverage. Further recommendations are provided in [appendix F](#).



We recommend that Propeller Heads incorporate more advanced testing techniques, such as fuzzing. As already mentioned, Propeller Heads plans to replace uses of `panic`, `unwrap`, and `expect` for recoverable errors. Fuzzing could help in assessing whether such an effort is complete. That is, Propeller Heads could fuzz arbitrary functions and note the ones that panic unexpectedly. Further recommendations on this topic are provided in [appendix G](#).

Finally, there are several places in the Rust code where token pairs are required to be sorted (e.g., the first token address must be smaller than the second). However, we could not identify a location common to all such paths. Hence, we recommend enforcing this constraint with a Rust type. That is, we recommend adding a type `SortedTokenPair`, ensuring that it is sorted when it is constructed, ensuring it cannot be changed thereafter, and using this type wherever it is required that a token pair's addresses be sorted.

Recommendations

Based on the findings identified during the security review, Trail of Bits recommends that Propeller Heads take the following steps:

- **Remediate the findings disclosed in this report.** These findings should be addressed through direct fixes or broader refactoring efforts.
- **Resolve all of the known issues and implement all planned improvements.** [Appendix C](#) contains 16 known issues and planned improvements that Propeller Heads provided to us at the start of the audit. We recommend addressing them before putting Turbine into production.
- **Complete the rewrite of the Speedex binary in Rust.** Currently, Turbine uses a Speedex binary written in C++. However, we understand that implementing Speedex in Rust would facilitate attestation from Phala Cloud. We recommend completing the translation for this reason.
- **Fix the project's tests.** In the commit that we reviewed, running `cargo test` fails because some tests require environment variables to hold privileged contents. Tests should fail only when the code is not working. In its current state, the project leaves users with the incorrect impression that the code is broken. We recommend addressing this problem so that users are encouraged to run the tests and to find and report bugs in the codebase.
- **Regularly compute the Solidity and Rust code's test coverage.** We computed coverage for the Solidity code using [forge coverage](#) and for the Rust code using [cargo-llvm-cov](#). For both, test coverage is considerably less than 100%. We recommend striving for 100% test coverage of the Solidity code, and as close to 100% as possible for the Rust code.

- **Incorporate more advanced testing strategies, such as fuzzing.** We found many panics by fuzzing arbitrary functions with `test-fuzz` and `cargo-afl`. We recommend continuing such a strategy to ensure that all recoverable panics have been identified and eliminated.
- **Incorporate a `SortedTokenPair` type and use it in Rust code that requires token addresses to be sorted.** Many parts of the Rust code require a token pair to be sorted. However, it is difficult to identify a single location common to all such paths. We recommend enforcing the sortedness constraint with a type, and using the type wherever the constraint is needed. Further recommendations appear in [appendix H](#).
- **Seek a reaudit.** Many of the changes described above are nontrivial. For example, completing the 16 known issues and planned improvements in [appendix C](#) will require considerable work. Hence, we recommend that Propeller Heads seek a reaudit of the Turbine codebase once all of the above changes are completed.

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

<i>Severity</i>	<i>Count</i>
High	3
Medium	6
Low	4
Informational	8
Undetermined	0

CATEGORY BREAKDOWN

<i>Category</i>	<i>Count</i>
Cryptography	1
Data Exposure	1
Data Validation	12
Denial of Service	2
Error Reporting	1
Testing	1
Timing	1
Undefined Behavior	2



Project Goals

The engagement was scoped to provide a security assessment of the Turbine trustless dark solver. Specifically, we sought to answer the following non-exhaustive list of questions:

- Is it possible to steal user funds?
- Could the spending approvals that users give to Turbine be abused by an attacker?
- Could an attacker manipulate the prices that Turbine uses to determine whether swaps are fair?



Project Targets

The engagement involved a review and testing of the following target.

Turbine

Repository	https://github.com/propeller-heads/turbine
Version	9644d88d5aee50a4f93feaeb646123b74730b015
Type	Rust
Platform	POSIX



Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level engagement goals. Our approaches included the following:

- **Documentation review.** We reviewed the Turbine documentation provided to us as a PDF.
- **Test coverage review.** We verified that the code builds and the tests pass. We also computed test coverage for the Solidity code using `forge coverage` and for the Rust code using `cargo-llvm-cov`.
- **Concurrency analysis.** We ran the Rust tests with `ThreadSanitizer` enabled to look for race conditions.¹
- **Static analysis.** We ran `Clippy`, `Dylint`, and `Semgrep` over the Rust code and reviewed the results. We ran `Slither` over the Solidity code and reviewed the results.
- **Fuzzing.** We fuzzed several Rust functions and methods, looking for easily discoverable panics. We augmented this with `cast_checks` to look for invalid casts.
- **Manual review.** We manually reviewed all Rust files except for `testing.rs`, which appeared to contain just testing code. We manually reviewed all Solidity code. A complete list of both types of files appears below.
 - Rust:
 - `abi.rs`
 - `api` directory
 - `block_subscription.rs`
 - `blockchain_gateway` directory
 - `clearing_algorithm` directory
 - `config.rs`
 - `constants.rs`
 - `controller.rs`
 - `dex_gateway` directory
 - `execution.rs`
 - `factory.rs`
 - `fees.rs`
 - `lib.rs`
 - `main.rs`

¹ ThreadSanitizer did not reveal any concurrency issues. But it did reveal that some tests are flaky. This motivated our recommendation in [appendix D](#) to limit the use of `sleep`.

- metrics directory
 - mid_price_calculator.rs
 - models.rs
 - price.rs
 - price_providers directory
 - provider_ext.rs
 - serializers.rs
 - settlement_trigger.rs
 - solution_submitter directory
 - state directory
 - tee.rs
 - utils.rs
 - validation.rs
- Solidity:
 - interfaces/IAAllowanceTransfer.sol
 - interfaces/IPermit2.sol
 - interfaces/ISignatureTransfer.sol
 - settler/Constants.sol
 - settler/TurbineSettler.sol
 - settler/interfaces/ITurbineLiquidityRouter.sol
 - settler/interfaces/ITurbineSettler.sol
 - settler/libraries/AccessControlled.sol
 - settler/libraries/Address.sol
 - settler/libraries/OrderLib.sol
 - settler/structs/BatchSettlement.sol
 - settler/structs/Execution.sol
 - settler/structs/Order.sol
 - settler/structs/OrderGroup.sol
 - settler/structs/Price.sol
 - univ4-hook/BaseHook.sol
 - univ4-hook/LPToken.sol
 - univ4-hook/README.md
 - univ4-hook/TurbineHook.sol
 - univ4-hook/TurbineLiquidityRouter.sol
 - univ4-hook/libraries/CurrencySettler.sol
 - univ4-hook/libraries/PoolMetadata.sol

Coverage Limitations

Because of the time-boxed nature of testing work, it is common to encounter coverage limitations. The following list outlines the coverage limitations of the engagement and indicates system elements that may warrant further review:



- As mentioned above, Propeller Heads provided us a list of 16 known issues and planned improvements ([appendix C](#)). We made a best effort to not report findings that would overlap with this list.
- We did not review the code for the Speedex binary.
- We did not review the code for the Tycho service.
- We ran Turbine under the TAPPD simulator but did not test it directly on Phala Cloud. At the time of the audit, there was no way to produce a complete, measured Phala Cloud deployment that included the Speedex binary.

Automated Testing

Trail of Bits uses automated techniques to extensively test the security properties of software. We use both open-source static analysis and fuzzing utilities, along with tools developed in-house, to perform automated testing of source code and compiled software.

Fuzzing

We used `test-fuzz` and `cargo-afl` to fuzz many Rust functions and methods. We held back on reporting panics for errors that appear to be recoverable. However, we did report panics observed in third-party dependencies:

- **TOB-TURBINE-3:** `PermitSingle` and `PermitBatch`'s `From` implementations can panic in `U48::from`
- **TOB-TURBINE-4:** `u256_to_naive_datetime` can produce values before UNIX epoch

To emphasize, fuzzing an intermediate function and observing a panic does not imply the function could panic from user-provided input.

A complete list of the functions that we fuzzed is provided in [appendix G](#).

Codebase Maturity Evaluation

Trail of Bits uses a traffic-light protocol to provide each client with a clear understanding of the areas in which its codebase is mature, immature, or underdeveloped. Deficiencies identified here often stem from root causes within the software development life cycle that should be addressed through standardization measures (e.g., the use of common libraries, functions, or frameworks) or training and awareness programs.

Category	Summary	Result
Arithmetic	The code uses checked arithmetic in several places but silences resulting errors with <code>unwrap_or</code> . Thus, this category could not be evaluated.	Further Investigation Required
Auditing	The Rust code uses tracing macros (e.g., <code>error!</code> and <code>warn!</code>) judiciously. Similarly, the Solidity code emits events that include pertinent information for the operations being reported on.	Satisfactory
Authentication / Access Controls	Propeller Heads acknowledged that there is currently no way to validate Turbine code running in a TEE. Additionally, the Turbine application relies on a Speedex binary. A quote returned from the Turbine TEE code does not measure the Speedex binary.	Further Investigation Required
Complexity Management	The code was accompanied by a list of 16 known issues and planned improvements (appendix C). Implementing many of those improvements will be nontrivial as they require API changes. The documentation and code contain several “TODOs.”	Further Investigation Required
Cryptography and Key Management	While cryptography and key management do not feature prominently in this codebase, the code contains at least one flaw related to ECDSA signature malleability (TOB-TURBINE-11).	Moderate
Decentralization	Propeller Heads relies on one service (Phala Cloud) to run its TEE. During the first week of the audit, a CloudFlare outage occurred. The outage made it impossible to get quotes for their existing deployment. During a	Weak

	subsequent week of the audit, we were unable to run cargo test because the Tycho service was down.	
Documentation	While the existing documentation is clear and helpful, it has many “TODOs” and glaring omissions. For example, the term “smart orders” appears throughout the code. However, there is no mention or discussion of “smart orders” in the documentation.	Moderate
Low-Level Manipulation	The Rust code does little bit- or byte-level manipulation and uses <code>unsafe</code> sparingly. The Solidity code uses inline assembly in two ways: to rethrow errors from lower-level calls and to compute pool IDs using less gas than would be required if the code were implemented in Solidity.	Satisfactory
Testing and Verification	The code’s tests rely on environment variables whose values require privilege to obtain. Some critical code is not executed by tests (see appendix F for more details). The code would benefit from more advanced testing techniques, such as fuzzing.	Weak
Transaction Ordering	We found no problems related to transaction ordering.	Satisfactory

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	Documentation for running tests needs to be updated	Testing	Informational
2	serialize_struct argument does not match number of serialize_field calls	Undefined Behavior	Informational
3	PermitSingle and PermitBatch's From implementations can panic in U48::from	Data Validation	Informational
4	u256_to_naive_datetime can produce values before UNIX epoch	Undefined Behavior	Low
5	Invalid bit-length comparison in reduce_precision	Data Validation	Medium
6	Recursive price filtering allows oracle threshold bypass	Denial of Service	Medium
7	SIWE message version field is not validated	Data Validation	Low
8	U256 arithmetic errors silenced with unwrap_or	Error Reporting	Informational
9	Authentication credentials leak when transmitted over unencrypted HTTP	Data Exposure	Medium
10	Potential integer overflow in function scale_up_input	Data Validation	High
11	ECDSA signature malleability	Cryptography	Low



12	Missing validation of user balance vector length	Data Validation	Low
13	Missing block number validation in pool state updates	Data Validation	Informational
14	Race conditions are present in filesystem operations	Timing	Informational
15	Unable to safely change hook address	Data Validation	High
16	Unbounded intent array enables denial of service of settlement system	Denial of Service	High
17	Nonstandard ERC-20 tokens cause swap failures due to unsafe transfer checks	Data Validation	Medium
18	Insufficient pre-execution validation enables single-user actions to cause batch settlement failures	Data Validation	Medium
19	Remove liquidity operations lack slippage protection, enabling unfavorable token ratios	Data Validation	Medium
20	Off-by-one-second discrepancy in speedbump enforcement between on-chain and off-chain paths	Data Validation	Informational
21	Missing minimum buy amount validation allows users to receive less than specified minimum	Data Validation	Informational

Detailed Findings

1. Documentation for running tests needs to be updated

Severity: Informational

Difficulty: Low

Type: Testing

Finding ID: TOB-TURBINE-1

Target: README.md

Description

Cloning the repository and running cargo test can cause problems in two ways, which are documented in this finding.

First, some tests expect the following environment variables to be set:

- READ_PROVIDER_RPC_URL: Ethereum RPC provider URL
- TYCHO_AUTH_KEY: Tycho API key

Simply cloning the repository and running cargo test will fail because these environment variables are not set. To obtain a Tycho API key, one must register with Tycho. To obtain an Ethereum RPC URL, one must register with a service such as Alchemy. The tests that require these environment variables should be feature-gated so that simply running cargo test succeeds.

Second, some tests modify production files. Specifically, the test_main_loop test modifies the data/backup/orderbook.json file. After we reported this issue, Propeller Heads produced the fix shown in figure 1.1 to eliminate the test's modification of the file. (At the time this finding was written, this fix had not yet been merged.)

```
diff --git a/src/controller.rs b/src/controller.rs
index d5fa7c97..cbec414b 100644
--- a/src/controller.rs
+++ b/src/controller.rs
@@ -969,6 +969,7 @@ mod tests {
     use alloy::primitives::{Address, B256, Bytes, U256, aliases::U48};
     use chrono::TimeDelta;
     use indexmap::IndexMap;
+    use tempfile::TempDir;
     use tokio::sync::RwLock;
```



```

    use super::*;
@@ -1024,10 +1025,9 @@ mod tests {
    let liquidity_manager = LiquidityManager::default();
    let validator = Validator::new(blockchain_gateway.clone(),
&TurbineConfig::default());
    let fee_calculator = FeeCalculator::new(&config);
-    let state_gateway = create_state_gateway(
-        Backend::File(config.state_backup_path.clone().into()),
-        config.clone(),
-    );
+    let temp_dir = TempDir::new().unwrap();
+    let state_gateway =
+        create_state_gateway(Backend::File(temp_dir.path().to_path_buf()),
config.clone());
    let protocol_state = Arc::new(ProtocolState::new(
        Address::ZERO,
        orderbook,

```

Figure 1.1: Fix to eliminate test_main_loop's modification of production files

Exploit Scenario

Alice is an Ethereum advocate. She considers using Turbine. She clones the Turbine repository and runs cargo test, which fails. She is dissuaded by the failure and chooses another lending solution.

Recommendations

Short term, take the following steps:

- Gate tests that use environment variables behind Cargo feature gates. Doing so will prevent users from observing test failures unrelated to code errors.
- Merge the fix shown in figure 1.1, if it has not been merged already. Doing so will prevent the test_main_loop test from modifying the data/backup/orderbook.json file.

Long term, regularly run the project's tests without any credentials. Doing so will alert you to problems that users may encounter when they try to run the tests.

2. `serialize_struct` argument does not match number of `serialize_field` calls

Severity: Informational

Difficulty: Undetermined

Type: Undefined Behavior

Finding ID: TOB-TURBINE-2

Target: `src/state/gateway/file.rs`

Description

`LiquidityManager`'s `serialize` implementation calls `serialize_struct` with a `len` argument that is not equal to the number of fields that it serializes (figure 2.1). Serde's documentation is ambiguous on this point:

The name is the name of the struct and the len is the number of data fields that will be serialized. len does not include fields which are skipped with `SerializeStruct::skip_field`.

That is, the documentation is unclear as to whether a mismatch between the `len` argument and the number of fields serialized can cause harm. We suspect it depends on the `Serializer` involved (e.g., `serde_json::Serializer`). To avoid potential harm, we recommend changing `len` to 2.

```
70     impl Serialize for LiquidityManager {
71         fn serialize<S>(&self, serializer: S) -> Result<S::Ok, S::Error>
72         where
73             S: serde::Serializer,
74         {
75             use serde::ser::SerializeStruct;
76
77             let mut state = serializer.serialize_struct("LiquidityManager", 3)?;
78             state.serialize_field("actions", &self.actions)?;
79             state.serialize_field("intent_history", &self.intent_history)?;
80             state.end()
81         }
82     }
```

Figure 2.1: `LiquidityManager`'s `Serialize` implementation. Note that `serialize_struct`'s `len` argument is 3, but that the call is followed by two calls to `serialize_field`. ([turbine/src/state/gateway/file.rs#70-82](#))

Exploit Scenario

Alice is an Ethereum developer. She tries to port Turbine to work with a different serialization format than JSON. A bug in Alice's implementation causes her port to crash



when `state.end()` is called on line 80 of `src/state/gateway/file.rs`, shown in figure 2.1.

Recommendations

Short term, change `serialize_struct`'s `len` argument from 3 to 2. Doing so will reduce the likelihood that a port of Turbine to another serialization format will result in crashes.

Long term, consider running Dylint's `wrong_serialize_struct_arg` lint regularly. This bug was found by running the lint on the codebase.

3. PermitSingle and PermitBatch's From implementations can panic in U48::from

Severity: Informational

Difficulty: Undetermined

Type: Data Validation

Finding ID: TOB-TURBINE-3

Target: src/api/signature.rs

Description

PermitBatch's From implementations can panic when they call `U48::from`. According to Rust's documentation, From implementations should not panic (emphasis preserved from the original):

Note: This trait must not fail. The From trait is intended for perfect conversions. If the conversion can fail or is not perfect, use TryFrom.

The relevant code appears in figures 3.1 through 3.3. The issue arises when a 64-bit value is cast to a 48-bit value. As can be seen from the code in figure 3.4, `ruint`'s code can clearly panic.

```
153     #[derive(Debug, Serialize, Deserialize, ToSchema, Clone)]
154     pub struct PermitDetails {
155         #[schema(value_type = String, format = "address", example =
156         "0xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48")]
157         pub token: Address,
158         #[schema(value_type = String, format = "uint160", example =
159         "1461501637330902918203684832716283019655932542975")]
160         pub amount: U160,
161         #[schema(value_type = u64, format = "uint64", example = "1630003600")]
162         pub expiration: u64,
163         #[schema(value_type = u64, format = "uint64", example = "0")]
164         pub nonce: u64,
165     }
```

Figure 3.1: Definition of PermitDetails ([turbine/src/api/models.rs#153-166](#))

```
32     impl From<PermitSingle> for IAllowanceTransfer::PermitSingle {
33         fn from(permit: PermitSingle) -> Self {
34             IAllowanceTransfer::PermitSingle {
35                 details: IAllowanceTransfer::PermitDetails {
```



```

36         token: permit.details.token,
37         amount: permit.details.amount,
38         expiration: U48::from(permit.details.expiration),
39         nonce: U48::from(permit.details.nonce),
40     },
41     spender: permit.spender,
42     sigDeadline: permit.sig_deadline,
43 }
44 }
45 }

```

Figure 3.2: PermitSingle's From<PermitSingle> implementation
([turbine/src/api/signature.rs#32-45](#))

```

105 impl From<PermitBatch> for IAllowanceTransfer::PermitBatch {
106     fn from(permit: PermitBatch) -> Self {
107         IAllowanceTransfer::PermitBatch {
108             details: permit
109                 .details
110                 .iter()
111                 .map(|detail| IAllowanceTransfer::PermitDetails {
112                     token: detail.token,
113                     amount: detail.amount,
114                     expiration: U48::from(detail.expiration),
115                     nonce: U48::from(detail.nonce),
116                 })
117                 .collect(),
118             spender: permit.spender,
119             sigDeadline: permit.sig_deadline,
120         }
121     }
122 }

```

Figure 3.3: PermitBatch's From<PermitBatch> implementation
([turbine/src/api/signature.rs#105-141](#))

```

160 pub fn from<T>(value: T) -> Self
161 where
162     Self: UintTryFrom<T>,
163 {
164     match Self::uint_try_from(value) {
165         Ok(n) => n,
166         Err(e) => panic!("Uint conversion error: {e}"),
167     }
168 }

```

Figure 3.4: ruint's panicking From implementation ([uint/src/from.rs#160-168](#))

Exploit Scenario

Mallory finds a way to send a PermitBatch with an expiration larger than 2^{48} to PermitBatch::from. The function panics, causing Turbine to crash.



Recommendations

Short term, refactor the code to check whether the 64-bit value will fit into 48 bits before performing the conversion. Doing so will prevent `ruint` from producing a panic.

Long term, avoid calling panicking functions from code that should not panic. `ruint`'s documentation states clearly that its `From` implementation can panic.

4. u256_to_naive_datetime can produce values before UNIX epoch

Severity: Low

Difficulty: Low

Type: Undefined Behavior

Finding ID: TOB-TURBINE-4

Target: src/state/liquidity_provision/models.rs

Description

The `u256_to_naive_datetime` function converts a U256 value to a u64 value, which it then casts to an i64 value. Casting from a u64 to an i64 in Rust is undefined behavior when the value is 2^{63} or larger. Experiments suggest that when that value wraps and is negative, `DateTime::from_timestamp` returns a value before the UNIX epoch.

```
251 fn u256_to_naive_datetime(value: U256) -> NaiveDateTime {
252     let seconds = value
253         .min(U256::from(u64::MAX))
254         .to::<u64>();
255     DateTime::from_timestamp(seconds as i64, 0)
256         .unwrap_or_else(|| DateTime::from_timestamp(0, 0).unwrap())
257         .naive_utc()
258 }
```

Figure 4.1: Definition of `u256_to_naive_datetime`
([turbine/src/state/liquidity_provision/models.rs#251-258](#))

Exploit Scenario

Mallory finds a way to pass exceedingly large seconds values to Turbine's `u256_to_naive_datetime` function. The chrono implementation converts the values to u64s and then casts them to i64s, which appear negative because their high bits are set.

Recommendations

Short term, refactor `u256_to_naive_datetime` so that it can return an error if its inputs are invalid. As shown above, `DateTime::from_timestamp` function does not treat such values as invalid.

Long term, consider testing with `cast_checks` regularly. This bug was found with `cast_checks`.



5. Invalid bit-length comparison in reduce_precision

Severity: **Medium**

Difficulty: **High**

Type: Data Validation

Finding ID: TOB-TURBINE-5

Target: src/price.rs

Description

The `reduce_precision` function in the file `price.rs` contains an off-by-one error in its bit-length comparison that leads to erroneous results and allows for the creation of invalid prices with zero denominators. The function is intended to panic when the difference between the numerator and denominator bit lengths exceed 256 bits, preventing prices that cannot be accurately represented in U256. However, the comparison uses a strict greater-than operator (`>`) instead of a greater-than-or-equal-to operator (`>=`), allowing edge cases where the bit-length difference equals exactly 256 bits to pass through unchecked.

```
// Reduces the ratio to fit 256 bits losing precision
fn reduce_precision(numerator: U512, denominator: U512) -> (U256, U256) {
    if numerator
        .bit_len()
        .abs_diff(denominator.bit_len()) >
        U256::BITS
    {
        // Difference in bit length is too large to reduce to 256 bits
        panic!("Price ratio cannot fit in 256 bits");
    }

    let max_bits = numerator
        .bit_len()
        .max(denominator.bit_len());
    let shift_bits = max_bits.saturating_sub(U256::BITS);
    (U256::from(numerator >> shift_bits), U256::from(denominator >> shift_bits))
}
```

Figure 5.1: `turbine/src/price.rs#L62-L79`

When the bit-length difference equals exactly 256, the function proceeds to calculate a shift amount that causes the smaller value to be shifted beyond its entire bit width, resulting in a zero value. For example, providing a numerator of 2^{256} (257 bits) and a denominator of 1 (1 bit) creates a 256-bit difference. The function calculates `max_bits` as 257 and `shift_bits` as 1, which shifts the denominator right by 1 bit. Since the denominator is 1, this shift results in 0, which is mathematically incorrect.



Exploit Scenario

An attacker provides carefully crafted numerator and denominator values to the function `from_u512_ratio`, which calls `reduce_precision`. The `reduce_precision` function accepts these values because the bit-length difference (256) is not strictly greater than 256 and shifts both values right by 1 bit, producing a numerator of 2^{255} and a denominator of 0. This price triggers a panic in the call to `Price::new` in `from_u512_ratio` since the denominator is zero.

Recommendations

Short term, change the comparison operator in the `reduce_precision` function from `>` to `>=` on line 66 of `src/price.rs`.

Long term, implement comprehensive validation in the `price.rs` source file to verify that all functions correctly handle edge cases. Consider adding fuzz tests that generate random U512 numerator and denominator pairs across the full range of bit lengths to verify that the precision reduction never produces zero values and maintains the ratio within acceptable bounds.

6. Recursive price filtering allows oracle threshold bypass

Severity: **Medium**

Difficulty: **Medium**

Type: Denial of Service

Finding ID: TOB-TURBINE-6

Target: `src/mid_price_calculator.rs`

Description

The `remove_outliers` function uses a recursive algorithm that can reduce the number of valid prices below the configured security threshold. The system requires a minimum of three oracle price sources to prevent manipulation, but the filtering algorithm suffers from two related flaws that allow an attacker to bypass this requirement.

First, the base case uses a hard-coded value of two instead of the configured minimum threshold, as can be seen in the code excerpt in figure 6.1.

```
fn remove_outliers(&self, prices: Vec<Price>) -> Vec<Price> {
    if prices.len() <= 2 {
        return prices;
    };
    let median = if let Ok(median) = prices.median() {
        median.to_f64_lossy()
    } else {
        error!("Failed to compute median while removing outliers");
        return vec![];
    };
};
```

Figure 6.1: `turbine/src/mid_price_calculator.rs#L132-L142`

Second, the recursive filtering algorithm itself enables cascade effects. An attacker who controls a single oracle can inject a strategic price value that triggers multiple filtering passes, progressively removing legitimate prices until fewer than the minimum threshold remain. Each recursive pass recalculates the mean and median from the reduced set, which can then cause previously valid prices to exceed deviation thresholds and be filtered out in subsequent passes. This creates a non-monotonic behavior where adding more data points reduces the number of valid outputs.

Recall the specific threshold filtering parameters used in Turbine, as defined in the documentation under the page [Market Price Oracle](#):



This price is the median market mid-price of independent price oracles. At least three oracles have to provide a price; if an oracle differs by more than 5% from the mean of all oracles, it's discarded; if the difference between mean and median of oracle prices is more than 1%, the most outlying oracle is discarded.

Thus, with a minimum threshold of three and a mean-median threshold of 1% (which is what the current implementation uses as thresholds), the legitimate price set [1901, 2000, 2099] passes validation and does not get filtered. However, if a malicious oracle injects a fourth price of 1920, the algorithm cascades: the first pass removes 2099 as an outlier, leaving [1901, 1920, 2000]. The second pass then removes 2000, leaving only [1901, 1920], which is below the three-oracle minimum. This causes the calling function `calculate_mid_price` to return a `MidPriceCalculatorError`, since the list of prices is below the oracle threshold.

Exploit Scenario

An attacker controls one price oracle. The three honest oracles report prices similar to the ones outlined in the finding description (i.e., 1901, 2000, 2099), which form a valid set that does not get filtered out in the `remove_outliers` call and meets the three-oracle minimum. The attacker strategically sends a price equal to 1920. When the recursive filtering algorithm processes these prices, it ends up removing two prices, which is below the desired threshold, and the calling function returns a `MidPriceCalculatorError`, preventing successful execution of the order.

Recommendations

Short term, add comprehensive documentation preceding the `remove_outliers` function to indicate its purpose and the reason for keeping it threshold-agnostic.

Long term, consider simplifying the filtering algorithm to prevent the cascading effect described above.

7. SIWE message version field is not validated

Severity: Low

Difficulty: Low

Type: Data Validation

Finding ID: TOB-TURBINE-7

Target: `src/api/validation.rs`

Description

The `verify_siwe` function performs a comprehensive validation of Sign-In with Ethereum (SIWE) authentication messages but does not verify the version field. According to the SIWE specification ([EIP-4361](#)), the version field *must* be set to 1, representing the current version of the message format.

```
pub fn verify_siwe(
    message: &siwe::Message,
    signature: &Signature,
    domain: &Authority,
    uri: &UriString,
    chain_id: u64,
    nonce: &String,
) -> Result<(), SiweVerificationError> {
    // Validate domain
    if message.domain != *domain {
        return Err(SiweVerificationError::DomainMismatch {
            expected: domain.to_string(),
            actual: message.domain.to_string(),
        });
    }
}
```

Figure 7.1: [turbine/src/api/validation.rs#L543-L557](#)

The current specification, under section [Message Fields](#), states the following:

version REQUIRED. The current version of the SIWE Message, which MUST be 1 for this specification.

Thus, the current SIWE specification defines version 1 as the only valid version. While the version field does not directly impact the cryptographic security of the signature verification, it represents a deviation from the SIWE specification that could cause interoperability issues or unexpected behavior if future versions of SIWE introduce breaking changes to the message format.

Note that the implementation validates all the other message fields comprehensively, including required fields (`domain`, `address`, `uri`, `chain_id`, `nonce`, `issued_at`) and explicitly rejects optional fields (`expiration_time`, `not_before`, `request_id`, `resources`).

Exploit Scenario

A future version of the SIWE specification introduces a new version with different security semantics or message structure that causes Turbine to incorrectly process version 2 messages using version 1 validation logic. As a result, an attacker is able to bypass security controls or trigger unintended authentication failures.

Recommendations

Short term, add explicit validation of the message version field in the `verify_siwe` function. The code should check that `message.version` equals `siwe::Version::V1` and return a new error variant `VersionNotSupported` if it does not match.

Long term, regularly review the [ERC-4361 specification](#) for updates. Incorporate changes that require updates to the Turbine implementation. Doing so will help ensure the code continues to comply with the standard.

8. U256 arithmetic errors silenced with unwrap_or

Severity: Informational

Difficulty: High

Type: Error Reporting

Finding ID: TOB-TURBINE-8

Target: src/clearing_algorithm/models.rs,
src/clearing_algorithm/speedex/price_solver.rs,
src/state/orderbook.rs

Description

In at least six places, the code performs checked arithmetic on U256 values. However, if the operation overflows, the U256 value is replaced with some other value. Such replacement hides the intent of the operation and possibly introduces logic errors.

Figures 8.1 through 8.5 show the six locations identified, along with how we think they should be addressed.

```
46     self.gross_buy_amount
47         .checked_sub(self.buy_token_fees.total_fee_absolute())
48         .unwrap_or(U256::ZERO)
```

Figure 8.1: Use of checked_sub and unwrap_or that should be saturating_sub
([turbine/src/clearing_algorithm/models.rs#46-48](#))

```
210     *price = Price::new(self.base_precision, min_price)
211         .checked_mul(*price)
212         .unwrap_or(U256::ZERO);
```

Figure 8.2: Use of unwrap_or to silence an error that should be returned
([turbine/src/clearing_algorithm/speedex/price_solver.rs#210-212](#))

```
518     let remaining_amount = order
519         .sell_amount
520         .checked_sub(order.executed_sell_amount)
521         .unwrap_or(U256::ZERO);
```

Figure 8.3: Use of checked_sub and unwrap_or that should be saturating_sub
([turbine/src/state/orderbook.rs#518-521](#))

```
537     order
538         .sell_amount
539         .checked_div(remaining_amount)
```



```
540      .unwrap_or(U256::MAX);
```

*Figure 8.4: Use of `unwrap_or` to silence an error that should be returned
([turbine/src/state/orderbook.rs#537-540](#))*

```
608      let surplus_buy_amount = order_settlement
609          .buyAmount
610          .checked_sub(
611              order
612                  .limit_price
613                  .checked_mul(order_settlement.sellAmount) // Executed sell
614          amount (after fees)
615          .unwrap_or(U256::ZERO),
616      )
617      .unwrap_or(U256::ZERO);
```

Figure 8.5: Note that there are two uses of `unwrap_or` in this figure. The inner one should be removed, and the error returned. The outer one should be combined with `checked_sub` into `saturating_sub`. ([turbine/src/state/orderbook.rs#608-616](#))

Exploit Scenario

Mallory finds a way to call the code shown in figure 8.5 with a multiplication that overflows. The multiplication is replaced with `U256::ZERO`. Hence, the `surplus_buy_amount` is `order_settlement.buy_amount`. The discrepancy messes up the orderbook's accounting.

Recommendations

Short term, incorporate the recommendations in each of the captions for figures 8.1 through 8.5. Doing so will make clear when subtractions are allowed to saturate and how other types of overflows (e.g., multiplication overflows) should be handled.

Long term, make fuzzing a part of your testing strategy. Doing so will help to uncover bugs that could arise from silencing arithmetic errors.

9. Authentication credentials leak when transmitted over unencrypted HTTP

Severity: **Medium**

Difficulty: **Medium**

Type: Data Exposure

Finding ID: TOB-TURBINE-9

Target: `src/dex_gateway/tycho.rs`

Description

The `initialize` function in the TychoGateway implementation allows authentication credentials to be transmitted over unencrypted HTTP connections. The issue stems from the interaction between the `TychoStreamBuilder` method calls and the underlying implementation in the `tycho-client` library. The code determines whether to use TLS based on the URL scheme (the first highlighted line in figure 9.1). Subsequently, the implementation overrides the `no_tls` parameter by calling `no_tls` (the second highlighted line).

```
// We assume tycho_url has either https or http scheme.  
let no_tls = tycho_url.scheme() != "https";  
// TychoStreamBuilder expects a host string, not a full URL  
let tycho_url = tycho_url.host_str().unwrap();  
  
debug!("Initializing Tycho client with {} tokens with TLS: {}", tokens.len(),  
!no_tls);  
  
let (handle, mut stream) = TychoStreamBuilder::new(tycho_url, Chain::Ethereum)  
    .auth_key(Some(tycho_auth_key.into())) // auth_key() overrides no_tls to false  
    .no_tls(no_tls)
```

Figure 9.1: [turbine/src/dex_gateway/tycho.rs#L245-L254](#)

The vulnerability arises from the implementation of these builder methods in the `tycho-client` library. The `auth_key` method sets the internal `no_tls` flag to false to enforce TLS when credentials are present, presumably to prevent a leak of the authentication key:

```
/// Sets the API key for authenticating with the Tycho server.  
///  
/// Optionally you can set the TYCHO_AUTH_TOKEN env var instead. Make sure to set  
no_tsl  
/// to false if you do this.  
pub fn auth_key(mut self, auth_key: Option<String>) -> Self {  
    self.auth_key = auth_key;
```



```

    self.no_tls = false;
    self
}

/// Disables TLS/SSL for the connection, using `http` and `ws` protocols.
pub fn no_tls(mut self, no_tls: bool) -> Self {
    self.no_tls = no_tls;
    self
}

```

*Figure 9.2: The `no_tls` method unconditionally overrides the flag.
([tycho-indexer/src/stream.rs#L172-L186](#))*

Note that the `no_tls` function is slightly ambiguous and is actually a toggle to enable or disable TLS, instead of a function that unconditionally disables TLS.

This means that when the URL uses the `http://` scheme, TLS is initially enabled when Turbine calls `auth_key`, but then it is overridden and disabled with the call to `no_tls` with a `true` Boolean parameter. Thus, the authentication key is transmitted in plaintext over an unencrypted connection, exposing it to network eavesdropping and man-in-the-middle attacks. Notably, the documentation in the `auth_key` method explicitly warns about this issue (“Make sure to set `no_tls` [sic] to false if you do this”), but the builder pattern does not enforce this constraint.

Exploit Scenario

An operator configures Turbine with an HTTP Tycho URL (either through misconfiguration or under the assumption that an internal network does not require encryption). An attacker who has obtained network access through a compromised host, network tap, or control over network infrastructure captures the unencrypted traffic between Turbine and the Tycho server. The attacker extracts the authentication key from the plaintext WebSocket upgrade request and uses it to access the Tycho service directly.

Recommendations

Short term, add validation in the `initialize` function to reject HTTP URLs when an authentication key is provided. The code should check if `tycho_url.scheme() == "http"` and `tycho_auth_key` is non-empty, and return an error if so. This will prevent the misconfiguration from allowing plaintext credential transmission.

Long term, consider updating the `tycho-client` library to modify the builder pattern to enforce the security invariant. The `no_tls` method should return an error or panic if called with `true` when an authentication key has been set.

10. Potential integer overflow in function `scale_up_input`

Severity: **High**

Difficulty: **High**

Type: Data Validation

Finding ID: TOB-TURBINE-10

Target: `src/clearing_algorithm/speedex/utils.rs`

Description

The `scale_up_input` function multiplies user-provided token amounts by powers of 10 without checking for overflow. When the input amount is large enough, the multiplication silently wraps around the U256 maximum value, producing an incorrect result that is orders of magnitude smaller than expected. This corrupted value is then used by the Speedex clearing algorithm to compute trade execution prices, potentially leading to incorrect settlements.

The vulnerable multiplication is highlighted in the code excerpt in figure 10.1:

```
pub fn scale_up_input(amount: U256, decimals: u32) -> U256 {
    if decimals > SPEEDEX_DECIMAL_PRECISION {
        panic!("Decimals must be less than BASE_DECIMALS");
    }
    amount * U256::from(10u64.pow(SPEEDEX_DECIMAL_PRECISION - decimals))
}
```

Figure 10.1: `turbine/src/clearing_algorithm/speedex/utils.rs#L14-L19`

The function scales token amounts to a uniform 18-decimal precision required by the Speedex algorithm. For tokens with fewer decimals (such as USDC with 6 decimals), the scaling factor is 10^{12} . When `amount *  $10^{12}$` exceeds `U256::MAX`, the multiplication wraps around modulo 2^{256} , producing a much smaller value. Unlike primitive Rust integers, which panic on overflow in debug mode, U256 from the alloy library always wraps in both debug and release modes, providing no runtime safety net during development.

This pattern of unchecked multiplication appears in other locations throughout the codebase. In `src/state/liquidity_provision/models.rs`, the `calculate_withdrawn_amounts` function multiplies pool reserves by LP token amounts without overflow checks:

```
let amount0_out = (U256::from(reserve0) * lp_token_burnt /
```



```
lp_token_total_supply).to::<u128>();
let amount1_out = (U256::from(reserve1) * lp_token_burnt /
lp_token_total_supply).to::<u128>();
```

Figure 10.2: *turbine/src/state/liquidity_provision/models.rs#L548–L549*

Similarly, the function `apply_add_liquidity_action` in the same file performs unchecked multiplications when calculating required token amounts and LP tokens to mint:

```
if provided_ratio < reserves_ratio {
    concretized_token0 = action.token0_amount;
    concretized_token1 =
        (provided_token0_u256 * reserve1_u256 / reserve0_u256).to::<u128>();
} else {
    concretized_token1 = action.token1_amount;
    concretized_token0 =
        (provided_token1_u256 * reserve0_u256 / reserve1_u256).to::<u128>();
}
liquidity_increment =
    self.lp_token_supply() * U256::from(concretized_token1) / reserve1_u256;
```

Figure 10.3: *turbine/src/state/liquidity_provision/models.rs#L348–L358*

Exploit Scenario

An attacker submits a limit order to sell USDC with a sell amount of $U256::MAX / 10^{12} + 1,000,000$. When Turbine processes this order for clearing, `scale_up_input` is called to normalize the amount to 18 decimals. The multiplication overflows, wrapping the scaled amount to approximately 1 million in 18-decimal units. The Speedex clearing algorithm processes this accordingly, leading to incorrect results.

Recommendations

Short term, replace all unchecked multiplications with `checked_mul` and return `Result` types, propagating errors up the call stack.

Long term, add input validation at the API boundary to reject amounts that could overflow in any arithmetic operation.

11. ECDSA signature malleability

Severity: Low

Difficulty: Low

Type: Cryptography

Finding ID: TOB-TURBINE-11

Target: `src/api/signature.rs`

Description

The signature validation implementation in Turbine does not enforce the [EIP-2](#) requirement that ECDSA signature `s` values must be in the lower half of the `secp256k1` curve order. This allows an attacker to create an equivalent but distinct signature for any valid permit without possessing the private key.

The vulnerability exists in the `recover_address_from_prehash` method, which is called from the function `validate_signature` (figure 11.1).

```
pub fn validate_signature(
    &self,
    signature: &PermitSignature,
    owner: Address,
) -> Result<(), SignatureValidationError> {
    let hash = self.eip712_signing_hash(&PERMIT2_DOMAIN);
    let signer = signature.recover_address_from_prehash(&hash)?;
    if signer != owner {
        Err(SignatureValidationError::PermitSignerError { expected: owner, actual:
signer })
    } else {
        Ok(())
    }
}
```

Figure 11.1: [turbine/src/api/signature.rs#L78-L90](#)

The function `recover_address_from_prehash` is excerpted in figure 11.2, for reference.

```
pub fn recover_address_from_prehash(
    &self,
    hash: &B256,
) -> Result<Address, SignatureValidationError> {
    let signature = Signature::new(self.r, self.s, self.y_parity);
    let signer = signature.recover_address_from_prehash(hash)?;
    Ok(signer)
}
```



Figure 11.2: *turbine/src/api/signature.rs#L179–L186*

For any valid ECDSA signature (r, s, v) , an attacker can construct an alternative signature (r, s', v') where $s' = n - s$ (with n being the secp256k1 curve order) and v' is the flipped y-parity value. Both signatures recover to the same signer address and pass validation but have different byte representations.

Exploit Scenario

A user creates and signs a permit for a trade on Turbine with signature (r, s, v) . An attacker replays the trade and attaches the signature $(r, n - s, v')$ where n is the secp256k1 curve order. The attacker submits the trade with the modified signature to Turbine. Both signatures validate successfully as they recover to the same address. If Turbine later implements signature-based replay protection by tracking used signatures in a hashset, the attacker can submit orders with malleable signatures that bypass this protection.

Recommendations

Short term, add validation in the `recover_address_from_prehash` method to reject signatures where s exceeds half the curve order.

Long term, add comprehensive unit tests to exercise such edge cases.

12. Missing validation of user balance vector length

Severity: Low

Difficulty: High

Type: Data Validation

Finding ID: TOB-TURBINE-12

Target: `src/blockchain_gateway/rpc.rs`

Description

The `query_balances`, `query_erc20_allowances`, and `query_permit2_permits` functions use Ethereum multicall to batch queries efficiently. After executing batched RPC calls via `multicall.aggregate`, results are mapped back using the Rust `zip` operator, without validating that the number of results matches the number of requests.

```
// Build the result HashMap
let mut user_balances = UserBalances::new();
for ((user, token), balance) in user_token_pairs.iter().zip(all_results) {
    user_balances
        .entry(*user)
        .or_default()
        .insert(*token, balance);
}
```

Figure 12.1: *turbine/src/blockchain_gateway/rpc.rs#L120-L127*

The `zip` iterator terminates when the shortest input is exhausted. If the RPC returns fewer results than expected, extra user-token pairs are silently dropped without raising an error. This same pattern appears in the functions `query_erc20_allowances` and `query_permit2_permits` (figures 12.2 and 12.3).

```
317 // Build the result HashMap
318 let mut allowances = HashMap::new();
319 for ((owner, token), allowance) in owner_token_pairs
320     .iter()
321     .zip(all_results)
322 {
323     allowances
324         .entry(*owner)
325         .or_insert_with(HashMap::new)
326         .insert(*token, allowance);
327 }
```

Figure 12.2: *Use of zip in query_erc20_allowances*
(*turbine/src/blockchain_gateway/rpc.rs#317-327*)



```

370 // Build the result HashMap
371 let mut permits = HashMap::new();
372 for ((owner, token), result) in owner_token_pairs
373     .iter()
374     .zip(all_results)
375 {
376     let permit = PermitDetails {
377         token: *token,
378         amount: result.amount,
379         expiration: result.expiration,
380         nonce: result.nonce,
381     };
382     permits
383         .entry(*owner)
384         .or_insert_with(HashMap::new)
385         .insert(*token, permit);
386 }

```

*Figure 12.3: Use of zip in query_permit2_permits
([turbine/src/blockchain_gateway/rpc.rs#370-386](#))*

Exploit Scenario

A network error causes `multicall.aggregate` to return incomplete results. Turbine requests balances for 1,000 user-token pairs but receives only 950 responses. The `zip` operation processes 950 pairs and silently drops the remaining 50 without error. When Turbine validates an order from one of the 50 affected users, it finds no balance entry in the cache and incorrectly rejects a valid order or proceeds without proper balance validation. The silent failure makes debugging difficult, as no error is logged.

Recommendations

Short term, add validation that the result count matches the request count before user-token pairs are processed. This will prevent silent data loss and ensure all requested balances are retrieved.

Long term, test Turbine with randomly generated data from a mock RPC server. Doing so could help to expose bugs like this one.

13. Missing block number validation in pool state updates

Severity: **Informational**

Difficulty: **Undetermined**

Type: Data Validation

Finding ID: TOB-TURBINE-13

Target: `src/dex_gateway/tycho.rs`

Description

The `update_pool_states` function processes block updates from the Tycho stream and updates pool states accordingly. The function extracts the block number from the first protocol message in the update and sets `self.last_block` to this value without validating that the new block number is greater than or equal to the current `last_block`.

```
let block = block_update
    .state_msgs
    .values()
    .next()
    .unwrap()
    .header
    .number;

// ...

self.last_block = block;
```

Figure 13.1: [turbine/src/dex_gateway/tycho.rs#L94-L160](#)

Exploit Scenario

A network error causes the RPC provider to send a smaller (rather than larger) block number to Turbine. Turbine accepts the smaller block number without validating it. Turbine's accounting becomes inconsistent by referring to values from the past.

Recommendations

Short term, add validation that ensures incoming block numbers are monotonically increasing and rejects updates with regressed block numbers. This will prevent stale data from being accepted as current.

Long term, test Turbine with randomly generated data from a mock RPC server. Doing so could help to expose bugs like this one. (This is the same long-term recommendation provided for [TOB-TURBINE-12](#).)



14. Race conditions are present in filesystem operations

Severity: Informational

Difficulty: High

Type: Timing

Finding ID: TOB-TURBINE-14

Target: `src/clearing_algorithm/speedex/runner.rs`

Description

The Speedex runner writes a JSON blob to an input file, calls the Speedex binary, and then reads a file that contains the output produced by Speedex. This introduces two time-of-create-to-time-of-use (TOCTOU) race conditions.

```
let json_input = serde_json::to_string_pretty(speedex_input)?;
fs::write(&self.input_path, json_input)?;

let output = Command::new(&self.binary_path)
    .arg(&self.input_path)
    .arg(&self.output_path)
    .output()?;

if !output.status.success() {
    // ...snip...
}

// Read and parse the output file
let output_content = fs::read_to_string(&self.output_path)?;
let speedex_output: SpeedexOutput = serde_json::from_str(&output_content)?;
```

Figure 14.1: `turbine/src/clearing_algorithm/speedex/runner.rs#L251-L290`

The first race condition exists between `fs::write` (highlighted in yellow) and the invocation of the Speedex binary (highlighted in red). The second is between the Speedex binary's execution and reading of the output file (highlighted in green).

Typically, this type of race condition requires access to the filesystem of the Confidential VM to exploit. Based on our understanding of Phala Cloud and `dstack`, this is an unlikely prerequisite, so we consider this finding to be informational severity.

The random bytes used in the temporary directory path the files are created in are based on the `fastrand`, which is not a cryptographically secure random generator.

Exploit Scenario

Mallory is a malicious insider (Phala Cloud or Propeller Heads employee with privileged access). She deploys a program to monitor for filesystem changes and replace the JSON files with incorrect values. Despite TDX memory isolation, the Docker mount volume is accessible from the host, giving Mallory about 50 microseconds to monitor for file writes and inject manipulated orders.

Recommendations

Short term, update the scripts to write to a memory-mapped directory to avoid the latency of the filesystem.

Long term, consider not relying on an external Speedex binary.

15. Unable to safely change hook address

Severity: High

Difficulty: Low

Type: Data Validation

Finding ID: TOB-TURBINE-15

Target: contracts/src/univ4-hook/TurbineLiquidityRouter.sol

Description

The Turbine protocol contains a critical design flaw where all liquidity provider tokens become permanently frozen if the admin upgrades the hook contract. This occurs due to a synchronization failure between the mutable hook reference in `TurbineLiquidityRouter` and the immutable hook authorization stored in each `LPToken` contract.

Each `LPToken` contract stores the authorized hook address as a state variable during deployment. The `hookOnly` modifier restricts mint and burn operations exclusively to this stored address, with no mechanism to update it after deployment.

```
12     constructor(string memory _name, string memory _symbol, address _hook)
ERC20(_name, _symbol, 18) {
13         hook = _hook;
14     }
...
28     /// @notice Mints LP tokens. Can only be called by the authorized hook.
29     function mint(address to, uint256 amount) external hookOnly {
30         _mint(to, amount);
31     }
32
33     /// @notice Burns LP tokens. Can only be called by the authorized hook.
34     function burn(address from, uint256 amount) external hookOnly {
35         _burn(from, amount);
36     }
```

*Figure 15.1: LP token constructor sets the hook address permanently.
([turbine/contracts/src/univ4-hook/LPToken.sol#12-36](#))*

However, the `TurbineLiquidityRouter` contract provides an `updateHook` function (figure 15.2) that allows the admin to change the router's hook reference without any coordination with existing LP tokens. When this happens, the router will call the new hook to burn LP tokens during withdrawals, but the old LP tokens still require the original hook address. Since the new hook's address does not match the address stored in the LP token

contract, all burn calls revert with “Operation is restricted to the Turbine Hook,” permanently locking all liquidity in the pool.

```
145     function updateHook(address _hook) external onlyRole(DEFAULT_ADMIN_ROLE) {  
146         hook = TurbineHook(_hook);  
147         emit HookUpdated(_hook);  
148     }
```

Figure 15.2: Router allows hook updates with no LP token migration.

([turbine/contracts/src/univ4-hook/TurbineLiquidityRouter.sol#145–148](#))

Exploit Scenario

Alice, Bob, and hundreds of other users provide liquidity to multiple Turbine pools, receiving LP tokens representing their positions. The protocol operates normally for several months with total value locked reaching millions of dollars across all pools. The development team discovers a critical bug in the TurbineHook contract and decides to deploy a fixed version. The admin calls `updateHook` on the `TurbineLiquidityRouter`, pointing it to the new hook contract at address `0x2222`, while the old hook remains at address `0x1111`. Alice attempts to remove her liquidity by calling `removeLiquidity` on the router. The router calls the new hook’s `burn` function, which attempts to burn Alice’s LP tokens. The LP token contract checks `msg.sender` against its stored hook address: the new hook (`0x2222`) does not match the stored address (`0x1111`), causing the transaction to revert. Alice’s liquidity is permanently frozen, along with all other users’ funds.

Recommendations

Short term, modify LP tokens to query the router for the current hook address via `router.hook()` instead of storing it directly. This will maintain automatic synchronization after upgrades. Note that this change will be labor-intensive. Hence, Propeller Heads may want to skip straight to the long-term recommendation.

Long term, implement the TurbineHook contract as an upgradeable proxy using established patterns such as UUPS or a transparent proxy. This will allow the hook implementation to be upgraded while maintaining a constant proxy address that LP tokens reference. The proxy address remains unchanged across upgrades, eliminating the desynchronization issue entirely while providing a clean upgrade path. Additionally, add comprehensive integration tests that verify hook upgrades do not break liquidity operations, and include upgrade scenarios in the test suite to catch similar architectural issues.

16. Unbounded intent array enables denial of service of settlement system

Severity: High

Difficulty: Low

Type: Denial of Service

Finding ID: TOB-TURBINE-16

Target: contracts/src/univ4-hook/TurbineLiquidityRouter.sol

Description

The TurbineLiquidityRouter contract stores removal intents in an unbounded intentHashes array that must be fully iterated over on every settlement. The flushExpiredIntents function is called at the end of every settlement without gas limits or batching, creating a trivial denial-of-service attack vector. The RemoveLiquidityIntent struct includes a user-controlled salt field that allows for the creation of unique intent hashes.

```
22     struct RemoveLiquidityIntent {
23         /// @notice The address that owns the LP tokens and submits the intent.
24         address owner;
25         /// @notice The identifier of the pool from which liquidity should be
removed.
26         PoolId poolId;
27         /// @notice Amount of LP tokens to burn for withdrawal.
28         uint256 lpTokenAmount;
29         /// @notice Arbitrary user-provided salt to make the intent hash unique.
30         bytes32 salt;
31     }
```

Figure 16.1: Intent struct with user-controlled salt field

([turbine/contracts/src/settler/interfaces/ITurbineLiquidityRouter.sol#22-31](#))

Intent submission checks LP token balance but does not lock tokens, allowing an attacker to submit thousands of intents for as little as 1 wei each. Each intent with a different salt creates a unique hash, bypassing the duplicate check and allowing the intent to be appended to the intentHashes array.

```
157     function submitRemoveLiquidityIntent(
158         RemoveLiquidityIntent calldata intent,
159         AllowanceParams calldata permitParams
160     ) external returns (RemoveLiquidityIntentHash intentHash) {
    ...
199         intentHash = _hashRemoveLiquidityIntent(intent);
```



```

200     if (removeLiquidityIntents[intentHash].owner != address(0)) {
201         revert DuplicateIntent(intentHash);
202     }
    ...
207     intentHashes.push(intentHash); // Unbounded array growth
    ...
211 }

```

*Figure 16.2: Intent submission appends the intent to an unbounded array without cost.
([turbine/contracts/src/univ4-hook/TurbineLiquidityRouter.sol#157-211](#))*

The `flushExpiredIntents` function is called at the end of every settlement and must iterate through the entire array.

```

321 function flushExpiredIntents() external {
322     uint256 validUntil = block.timestamp;
323     uint256 intentsLength = intentHashes.length;
324     uint256 saveToIndex = 0;
325
326     for (uint256 inspectedSlot = 0; inspectedSlot < intentsLength;
inspectedSlot++) {
327         RemoveLiquidityIntentHash intentHash = intentHashes[inspectedSlot];
328         if (_isExpired(intentHash, validUntil)) {
329             _deleteIntentData(intentHash);
330             continue;
331         }
        ...
336     }
    ...
341 }

```

*Figure 16.3: Flush function iterates through the entire array on every settlement.
([turbine/contracts/src/univ4-hook/TurbineLiquidityRouter.sol#321-341](#))*

The function becomes impossible to execute once the array contains thousands of intents. Since `flushExpiredIntents` is called on every settlement, the protocol becomes permanently unable to execute settlements once this threshold is crossed. An attacker can submit thousands of intents with unique salt values. A smart contract can submit 5,000 intents in a single transaction, immediately disabling all settlements. Even after intents expire, the cleanup function cannot execute to remove them, creating a permanent denial of service.

Exploit Scenario

Alice deploys a smart contract that calls `submitRemoveLiquidityIntent` in a loop, submitting 5,000 removal intents with different salt values. All entries are added to the `intentHashes` array in one transaction. The next settlement batch calls `flushExpiredIntents`, which attempts to iterate through all 5,000 entries, exceeding the

block gas limit. The transaction reverts with an out-of-gas error. Every subsequent settlement attempt encounters the same revert, permanently disabling the protocol.

Recommendations

Short term, redesign the intent storage to use a mapping-based structure with off-chain indexing where the settler provides the list of intents to process, eliminating unbounded on-chain iteration. This will make gas consumption independent of total intent count.

Long term, establish development practices that avoid unbounded array iteration in critical paths. Add gas consumption analysis to the testing and review process to identify operations that scale linearly with user-controlled data structures.

17. Nonstandard ERC-20 tokens cause swap failures due to unsafe transfer checks

Severity: Medium

Difficulty: Low

Type: Data Validation

Finding ID: TOB-TURBINE-17

Target: contracts/src/univ4-hook/libraries/CurrencySettler.sol

Description

The CurrencySettler library wraps transfer and transferFrom calls in require statements that expect Boolean return values. However, major tokens like USDT do not return any value from their transfer functions, deviating from the ERC-20 standard. When Solidity attempts to decode an empty return value as a Boolean, the transaction reverts, making the protocol incompatible with USDT and similar nonstandard tokens even though USDT is explicitly whitelisted in the protocol configuration.

The vulnerable code wraps token transfers in require statements expecting Boolean returns (figure 17.1).

```
19  function settle(Currency currency, IPoolManager manager, address payer,
uint256 amount, bool burn) internal {
20      // for native currencies or burns, calling sync is not required
21      // short circuit for ERC-6909 burns to support ERC-6909-wrapped native
tokens
22      if (burn) {
23          manager.burn(payer, currency.toId(), amount);
24      } else if (currency.isAddressZero()) {
25          manager.settle{value: amount}();
26      } else {
27          manager.sync(currency);
28          if (payer != address(this)) {
29              require(
30                  IERC20Minimal(Currency.unwrap(currency)).transferFrom(payer,
address(manager), amount),
31                  "Transfer failed"
32              );
33          } else {
34              require(IERC20Minimal(Currency.unwrap(currency)).transfer(address(manager), amount),
"Transfer failed");
35          }
36          manager.settle();
37      }
```

Figure 17.1: Unsafe transfer calls expecting Boolean returns
([turbine/contracts/src/univ4-hook/libraries/CurrencySettler.sol#19–38](#))

This function is called during swap operations when settling input tokens to the PoolManager.

USDT's transfer function returns nothing, violating the ERC-20 standard that specifies a Boolean return value. When Solidity's ABI decoder attempts to decode empty bytes as a Boolean (32 bytes), it fails and reverts the transaction. This occurs even though USDT is explicitly whitelisted in the protocol configuration. The protocol inconsistently uses SafeERC20's safeTransfer in TurbineSettler for other operations but fails to use it in CurrencySettler, creating a critical incompatibility with the most widely traded stablecoin.

Exploit Scenario

Alice attempts to swap 1,000 USDT for WETH on a Turbine pool. The TurbineSettler calls swapExactTokens on the TurbineLiquidityRouter, which executes the swap and calls the swapCallback function. The callback attempts to settle the input tokens by calling inputCurrency.settle with USDT as the currency. The CurrencySettler.settle function calls USDT.transfer, wrapping it in a require statement that expects a Boolean return value. USDT's transfer function executes successfully and transfers the tokens, but returns empty bytes instead of a Boolean, reverting the transaction.

Recommendations

Short term, replace the unsafe transfer and transferFrom calls in CurrencySettler with OpenZeppelin's SafeERC20 library, which handles tokens that return nothing, return false, or return true. This matches the approach already used in TurbineSettler and will ensure compatibility with both standard and nonstandard ERC-20 implementations.

Long term, establish a consistent token interaction pattern across the entire codebase by mandating the use of SafeERC20 for all ERC-20 operations. Add integration tests with actual nonstandard tokens like USDT to verify compatibility before deployment. Document the list of supported tokens and their known deviations from the ERC-20 standard to prevent similar issues in future development.

18. Insufficient pre-execution validation enables single-user actions to cause batch settlement failures

Severity: Medium

Difficulty: Low

Type: Data Validation

Finding ID: TOB-TURBINE-18

Target: contracts/src/settler/TurbineSettler.sol

Description

The `executeLiquidityActionsAndSettlement` function processes all user actions atomically without failure isolation. The Rust back end validates actions only through post-batch simulation rather than checking critical on-chain conditions before batch construction. This allows a single invalid action to cause entire batch reverts, denying service to all users in that settlement.

```
470     function addLiquidity(  
471         address to,  
472         uint256 amount0,  
473         uint256 amount1,  
474         address poolToken0,  
475         address poolToken1,  
476         uint24 poolFee,  
477         uint256 liquidity,  
478         IAllowanceTransfer.PermitBatch memory tokensAllowanceParams,  
479         bytes memory permitSignature  
480     ) external payable turbineSettlerOnly returns (uint256) {  
481         if (amount0 == 0 || amount1 == 0 || liquidity <=  
hook.MINIMUM_LIQUIDITY()) {  
482             revert InsufficientLiquidityMinted();  
483         }  
         ...  
498     }
```

Figure 18.1: Minimum liquidity check enforced on-chain

([turbine/contracts/src/univ4-hook/TurbineLiquidityRouter.sol#470-498](#))

Critical failure conditions not validated in Rust include calculated liquidity below the 1,000 minimum, insufficient user token balances, used Permit2 nonces, insufficient allowances, LP token balances for removals, and nonstandard token transfer behaviors. Actions passing Rust validation can still revert on-chain due to state changes between validation and execution, causing batch-wide failures affecting all users.



Exploit Scenario

Bob submits a liquidity action to add 500 tokens to a pool, which passes Rust validation. The settlement system constructs a batch containing Bob's action plus 49 other valid user actions. The entire batch reverts with `InsufficientLiquidityMinted` due to Bob's failing transaction.

Recommendations

Short term, implement pre-execution validation that queries on-chain state for user balances and for `Permit2` allowances/nonces, and performs liquidity calculations to verify actions will succeed before batch inclusion. Add per-action simulation to detect and quarantine failing actions, preventing them from blocking valid user transactions.

Long term, redesign batch processing to isolate individual action failures using try-catch patterns or sub-calls, allowing valid actions to succeed even when others fail. Implement comprehensive integration tests ensuring all Solidity-level checks are replicated in Rust validation, and add monitoring for batch failure patterns to detect denial-of-service attempts.

19. Remove liquidity operations lack slippage protection, enabling unfavorable token ratios

Severity: Medium

Difficulty: Low

Type: Data Validation

Finding ID: TOB-TURBINE-19

Target: contracts/src/univ4-hook/TurbineLiquidityRouter.sol

Description

Liquidity removal intents calculate withdrawal amounts on-chain at execution time using current pool ratios, but provide no minimum output parameters for users to specify acceptable slippage tolerance. Between intent submission and execution, other liquidity operations in the queue can shift pool token ratios, causing users to receive more of one token and less of another than expected at submission time with no mechanism to reject unfavorable outcomes.

```
553     function _removeLiquidity(  
554         address to,  
555         uint256 liquidity,  
556         address poolToken0,  
557         address poolToken1,  
558         uint24 poolFee,  
559         AllowanceParams memory allowanceParamsLpToken  
560     ) internal returns (uint256, uint256) {  
561         ...  
572         (uint128 reserve0, uint128 reserve1) = hook.getReserves(poolToken0,  
poolToken1, poolFee);  
573         uint256 lpTokenTotalSupply = IERC20(hook.getLpTokenAddress(poolToken0,  
poolToken1, poolFee)).totalSupply();  
574         (uint256 amount0, uint256 amount1) =  
575             calculateWithdrawnAmounts(reserve0, reserve1, lpTokenTotalSupply,  
liquidity);  
576         // No minAmount0 or minAmount1 parameters to reject unfavorable ratios  
577         ...  
583     }
```

Figure 19.1: No slippage validation on-chain

([turbine/contracts/src/univ4-hook/TurbineLiquidityRouter.sol#553-583](#))

Standard AMMs prevent this with `minAmount0` and `minAmount1` parameters, allowing users to specify the minimum acceptable output for each token, and revert if the ratio becomes unfavorable.

Exploit Scenario

Alice holds 1,000 LP tokens and submits a remove liquidity intent when pool reserves are 100,000 USDC and 100 WETH. She expects to receive approximately 1,000 USDC and 1 WETH based on current ratios. Multiple other liquidity operations are queued before hers, including several large add liquidity operations that shift the pool ratio to 120,000 USDC and 95 WETH by the time Alice's removal executes. The `calculateWithdrawnAmounts` function at [TurbineLiquidityRouter.sol:574-575](#) computes her withdrawal as 1,200 USDC and 0.95 WETH using the execution-time reserves. Alice receives 20% more USDC but 5% less WETH than expected, an unfavorable trade if WETH price increased or if she needed specific token amounts. With no `minAmount0` or `minAmount1` parameters, Alice cannot reject this outcome and must accept the shifted ratio.

Recommendations

Short term, add `minAmount0` and `minAmount1` parameters to the `removeLiquidity` function signature and `RemoveLiquidityIntent` struct. Add on-chain validation of these bounds to `TurbineLiquidityRouter.sol` after withdrawal amounts are calculated, reverting if either token output falls below the user-specified minimum.

Long term, implement comprehensive slippage tests that verify protection works across various pool state changes during the delay window, including scenarios with multiple liquidity operations in the same batch. Consider allowing users to specify acceptable slippage as a percentage rather than absolute amounts for a better user experience.

20. Off-by-one-second discrepancy in speedbump enforcement between on-chain and off-chain paths

Severity: Informational

Difficulty: Low

Type: Data Validation

Finding ID: TOB-TURBINE-20

Target: contracts/src/univ4-hook/TurbineLiquidityRouter.sol

Description

The speedbump mechanism enforces a 12-second delay before liquidity removal intents become eligible for execution. An off-by-one-second discrepancy exists between on-chain intent execution and off-chain intent filtering: on-chain intents require strictly more than 12 seconds (`createdAt + 12 < block.timestamp`), while off-chain intents allow exactly 12 seconds (`created_at + 12 <= now`). This causes off-chain intents to become eligible 1 second earlier than on-chain intents.

```
704     function _isTooYoung(RemoveLiquidityIntentHash intentHash, uint256
createdBefore) internal view returns (bool) {
705         uint256 createdAt = removeLiquidityCreatedAt[intentHash];
706         if (createdAt >= createdBefore) {
707             return true;
708         }
709         return false;
710     }
```

Figure 20.1: No slippage validation on-chain

([turbine/contracts/src/univ4-hook/TurbineLiquidityRouter.sol#704-710](#))

```
266     let mut actions: Vec<LiquidityActionData> = self
267         .actions
268         .iter()
269         .filter(|a| a.created_at() <= max_created_at)
270         .cloned()
271         .collect();
```

Figure 20.2: No slippage validation off-chain

([turbine/src/state/liquidity_provision/manager.rs#266-271](#))

Off-chain removal intents execute via `removeLiquidity` directly without speedbump revalidation, while on-chain intents execute via `executePendingIntents`, which rechecks `_isTooYoung`. This creates a 1-second window where off-chain intents can execute but on-chain intents cannot.



The impact is minimal. The inconsistency is purely a precision difference in boundary conditions.

Recommendations

Short term, fix the Solidity implementation to match the documented 12-second requirement by changing `_isTooYoung` to use `>` instead of `>=`, making it `if (createdAt > createdBefore)`. This will align on-chain intent execution with the constant definition and off-chain behavior, allowing intents to execute at exactly 12 seconds as documented rather than requiring 13 seconds.

Long term, add explicit unit tests that verify intents become executable at exactly the 12-second boundary (not 13 seconds) for both on-chain and off-chain paths. Update code comments to clarify that `SPEEDBUMP_DURATION = 12 seconds` means “at least 12 seconds” (inclusive boundary), preventing future confusion about whether the check should be strict or inclusive.

21. Missing minimum buy amount validation allows users to receive less than specified minimum

Severity: Informational

Difficulty: Medium

Type: Data Validation

Finding ID: TOB-TURBINE-21

Target: contracts/src/settler/TurbineSettler.sol

Description

The `TurbineSettler` contract does not validate that users receive at least their specified minimum buy amount after fee deductions, violating a fundamental slippage protection guarantee expected in decentralized exchange protocols.

When settling limit orders, the contract calculates the total buy amount a user receives by adding their pro-rata surplus share to their minimum buy amount and then subtracting buy token fees.

However, when fees exceed the surplus allocation, users receive less than their specified minimum amount.

```
235     // Compute the surplus addressed to this user based on the volume fraction
they represent on buying
236     // this token.
237     uint256 userSurplus = 0;
238     for (uint256 j = 0; j < tokenAmounts.length; j++) {
239         if (tokenAmounts[j].token == swap.buyToken) {
240             userSurplus = tokenAmounts[j].surplus * swap.minBuyAmount /
tokenAmounts[j].expected;
241             break;
242         }
243     }
244
245     // The total amount the user will receive is computed as it's limit amount,
plus the surplus and discounted
246     // for the buy token fees.
247     uint256 totalBuyAmount = swap.minBuyAmount + userSurplus -
swap.buyTokenFees;
248
249     emit OrderSettled(
250         swap.owner, swap.orderHash, swap.sellToken, swap.buyToken,
swap.sellAmount, totalBuyAmount
251     );
252     IERC20(swap.buyToken).safeTransfer(swap.owner, totalBuyAmount);
```



*Figure 21.1: Vulnerable buy amount calculation without minimum validation
([turbine/contracts/src/settler/TurbineSettler.sol#235-252](#))*

The Solidity contract does not validate that `totalBuyAmount` is greater than or equal to `minBuyAmount` after the fee deduction, relying entirely on the off-chain Rust code to have correctly inflated the `minBuyAmount` parameter before settlement. The contract cannot verify that the inflation was performed correctly because it does not know the user's true minimum amount. The relationship between the Rust inflation logic and the Solidity fee deduction is not immediately apparent when reviewing the on-chain code. The Solidity contract does not validate that `totalBuyAmount` is greater than or equal to `minBuyAmount` after the fee deduction, and contains no documentation indicating that `minBuyAmount` represents a pre-inflated value.

Recommendations

Short term, add comprehensive documentation in both the Rust and Solidity codebases explaining that `minBuyAmount` represents a fee-inflated value rather than the user's true minimum. Include inline comments in `TurbineSettler.sol` that reference the corresponding Rust fee inflation implementation to make the relationship between off-chain inflation and on-chain deduction explicit.

Long term, consider refactoring the code to make this important logic easier to verify without requiring inspection of multiple files across different codebases. This could involve consolidating related logic, adding explicit validation comments at the calculation site, or restructuring the parameters to make the fee relationship more transparent.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	Description
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	Description
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	Description
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Maturity Categories

The following tables describe the code maturity categories and rating criteria used in this document.

Code Maturity Categories	
Category	Description
Arithmetic	The proper use of mathematical operations and semantics
Auditing	The use of event auditing and logging to support monitoring
Authentication / Access Controls	The use of robust access controls to handle identification and authorization and to ensure safe interactions with the system
Complexity Management	The presence of clear structures designed to manage system complexity, including the separation of system logic into clearly defined functions
Cryptography and Key Management	The safe use of cryptographic primitives and functions, along with the presence of robust mechanisms for key generation and distribution
Decentralization	The presence of a decentralized governance structure for mitigating insider threats and managing risks posed by contract upgrades
Documentation	The presence of comprehensive and readable codebase documentation
Low-Level Manipulation	The justified use of inline assembly and low-level calls
Testing and Verification	The presence of robust testing procedures (e.g., unit tests, integration tests, and verification methods) and sufficient test coverage
Transaction Ordering	The system's resistance to transaction-ordering attacks

Rating Criteria	
Rating	Description
Strong	No issues were found, and the system exceeds industry standards.
Satisfactory	Minor issues were found, but the system is compliant with best practices.
Moderate	Some issues that may affect system safety were found.
Weak	Many issues that affect system safety were found.
Missing	A required component is missing, significantly affecting system safety.
Not Applicable	The category does not apply to this review.
Not Considered	The category was not considered in this review.
Further Investigation Required	Further investigation is required to reach a meaningful conclusion.

C. Known Issues and Planned Improvements

This appendix contains a list of known issues and planned improvements provided by Propeller Heads. During the review, we intentionally avoided reporting findings on issues described in this appendix.

We would like to emphasize that the known issues listed below are not merely hypothetical; we encountered them during the review. Here are two examples:

- One known issue states that “there is no process for reproducible builds to prove that a certain Docker image was built from a certain code.” Note that running Turbine requires running a Speedex binary. Turbine’s build instructions suggest two ways of obtaining the binary: building it from scratch or downloading it from an Amazon S3 bucket. The latter method does not suggest a way to verify that the downloaded binary is the one that Propeller Heads uploaded.

Propeller Heads acknowledges that its method for producing TEE quotes does not measure the Speedex binary.

- A second known issue states that “when one price provider fails to instantiate, whole Turbine crashes.” We observed Turbine crashing with the following message after running some tests:

```
thread 'tokio-runtime-worker' (...) panicked at src/state/orderbook.rs:284:18:  
Mid price not found
```

We asked Propeller Heads about this, and the team determined that the problem was `test_main_loop` modifying `data/backup/orderbook.json`. This caused the team to produce the fix shown in figure 1.1 of [TOB-TURBINE-1](#).

Note that the lists below are taken verbatim from the document provided by Propeller Heads.

Rust Code

- Liquidity intents will use `SignatureTransfer` instead of `AllowanceTransfer`.
 - This means that nonces of permits attached to liquidity intents will not have to be verified (see the comment [here](#)).
- Market Maker quotes will be executed through `TychoRouter` contract, just as DEX swaps are executed now.
- `panic!`s, `unwraps` and `expects` will be replaced with error handling for all recoverable errors.



- There is no process for reproducible builds to prove that a certain Docker image was built from a certain code.

Price Providers

- When one price provider fails to instantiate, whole Turbine crashes.
- Chainlink, Binance and Pyth price providers will implement a `CachedPriceProvider` trait responsible to manage cached prices, including fetching, updating, and checking for staleness.
- Binance and Pyth price providers might have outdated prices.
- Binance and Pyth price providers read external APIs without sufficient security/sanity checks.
- Mid-prices from Propeller Price Provider are not symmetrical (i.e. mid-price for pair A/B is slightly different than inverted price for pair B/A)

Solidity Code

- In `TurbineSettler`, a new `TurbinePoolExecution` type will be extracted from `SmartOrderExecution` type. Instead of treating swaps on Turbine Pools like general smart orders, we'll have a dedicated type for them. `TurbineSettler` will call the `LiquidityRouter` contract explicitly.
- `CurrencySettler` solidity library will be removed.
- We will split off focused roles from `DEFAULT_ADMIN_ROLE`, so that not all permissions are given to a single wallet.
- There is no way to transfer protocol fees out of `TurbineSettler`
- Types in Solidity should be unified (e.g. we're using `u128` for pool reserves but `u256` for trade amounts)

Relevant to turbine-bots Repository

- `HashflowProxy` contract is missing accounting for the market maker surplus

D. Non-Security-Related Recommendations

This appendix contains recommendations that do not have immediate or obvious security implications. However, implementing them may enhance the code's readability and may prevent the introduction of vulnerabilities in the future.

- **Remove the `cargo install` command from both `README.md` and the `documentation`.** When run, the command produces the error shown in figure D.1.

```
$ cargo install
error: Using `cargo install` to install the binaries from the package in
current working directory is no longer supported, use `cargo install --path .`
instead. Use `cargo build` if you want to simply build the package.
```

Figure D.1: Error produced by running `cargo install`

- **Enable Clippy's `must_use_candidate` and `return_self_not_must_use` lints, and run them regularly.** When run, they produce many warnings (an example is shown in figure D.2).

```
warning: unused return value of `price::Price::new` that must be used
--> src/price.rs:365:9
365 |         Price::new(U256::ZERO, U256::from(1));
    |         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
help: use `let _ = ...` to ignore the resulting value
365 |         let _ = Price::new(U256::ZERO, U256::from(1));
    |         ++++++
```

Figure D.2: Sample warning produced when `#[must_use]` is added to `Price::new`

- **Remove the commented-out code shown in figure D.3.**

```
228 // return Err(OrderbookError::PermitNonceTooLow {
229 //     new: new_details.nonce.to_string(),
230 //     current: existing_details.nonce.to_string(),
231 // });
```

Figure D.3: Commented-out code that can be removed
([turbine/src/state/orderbook.rs#228-231](#))

- **In the code in figure D.4, change the `std::sync::RwLock` to a `tokio::sync::RwLock`.** Note that this change will require `set_cached_gas_price` and `get_cached_gas_price` to become `async`. Thus, their callers will have to change as well.

```
22    cached_gas_price_wei: RwLock<Option<u128>>, // latest gas price cached
    by controller
```

Figure D.4: Use of `std::sync::RwLock` (*turbine/src/fees.rs#22*)

```
1    /// Sets the cached gas price (wei) used for fee calculations.
2    pub async fn set_cached_gas_price(&self, gas_price_wei: u128) {
3        *self.cached_gas_price_wei.write().await = Some(gas_price_wei);
4    }
5
6    /// Returns the cached gas price (wei), if available.
7    pub async fn get_cached_gas_price(&self) -> Option<u128> {
8        *self.cached_gas_price_wei.read().await
9    }
```

Figure D.5: Proposed asynchronous versions of `set_cached_gas_price` and `get_cached_gas_price`

- **Reduce the tests' reliance on `tokio::time::sleep`.** Often, a better condition is available to wait on. Also, we noticed that the `test_propeller_provider_background_task` test fails sporadically, possibly because it relies on `tokio::time::sleep`.

```
src/api/router.rs:
tokio::time::sleep(Duration::from_millis(100)).await;
# ^ 30 more similar calls in src/api/router.rs
src/api/router.rs:
tokio::time::sleep(std::time::Duration::from_millis(50)).await;
src/api/router.rs:
tokio::time::sleep(tokio::time::Duration::from_secs(session_expiry_s as u64 +
1)).await;
src/api/session_store.rs:
tokio::time::sleep(tokio::time::Duration::from_secs(2)).await;
src/controller.rs:
tokio::time::sleep(tokio::time::Duration::from_millis(100)).await;
src/controller.rs:
tokio::time::sleep(tokio::time::Duration::from_millis(100)).await;
src/controller.rs:
tokio::time::sleep(tokio::time::Duration::from_millis(100)).await;
src/metrics/utils.rs:
tokio::time::sleep(Duration::from_millis(50)).await;
src/price_providers/binance.rs:
tokio::time::sleep(std::time::Duration::from_millis(1200)).await;
src/price_providers/chainlink.rs:
tokio::time::sleep(Duration::from_millis(1200)).await;
src/price_providers/propeller.rs:
tokio::time::sleep(Duration::from_millis(300)).await;
src/price_providers/propeller.rs:
tokio::time::sleep(Duration::from_millis(300)).await;
```

```
src/price_providers/propeller.rs:
tokio::time::sleep(Duration::from_millis(300)).await;
src/price_providers/pyth.rs:
tokio::time::sleep(std::time::Duration::from_millis(1200)).await;
src/settlement_trigger.rs:
tokio::time::sleep(duration_until_target.to_std().unwrap()).await;
src/solution_submitter/tenderly.rs:
tokio::time::sleep(tokio::time::Duration::from_secs(1)).await;
```

Figure D.6: Uses of `tokio::time::sleep` in the Turbine tests

- **Rewrite `MidPriceCalculator::remove_outliers` so that it is iterative rather than recursive.** Its uses of recursion appear in figure D.7.

```
158 // Mean and median difference cannot be more than
mean_median_threshold
159 if mean_median_difference > self.mean_median_threshold {
    ...
165 return self.remove_outliers(filtered_prices);
166 }
167
168 // No individual price can be further from the median than
max_price_deviation * median
169 let max_distance = self.max_price_deviation * median;
170 if prices_with_distance[0].2 > max_distance {
    ...
176 return self.remove_outliers(filtered_prices);
177 }
```

Figure D.7: Uses of recursion in `MidPriceCalculator::remove_outliers` ([turbine/src/mid_price_calculator.rs#158-177](#))

- **Eliminate the code's unconventional style of writing rustdoc comments.** An example appears in figure D.8. Instead, use the style of figure D.9. The latter style provides tooltips for IDEs that support them.

```
27 /// The Orderbook manages a collection of orders and provides access to
the blockchain through a
28 /// gateway.
29 ///
30 /// # Fields
31 /// * `orders` - An `IndexMap` mapping `OrderHash` to `Order`,
representing all active orders in the
32 /// orderbook, sorted from oldest to newest.
33 /// * `allowances` - A `HashMap` mapping token `Address` to another
`HashMap` from owner `Address`
34 /// to `AllowanceParams`, storing the latest tracked allowance for
each user/token combination to
35 /// prevent errors with nonce.
...

```

```
50 pub struct Orderbook {
```

Figure D.8: Unconventional style of writing rustdoc comments for struct fields
([turbine/src/state/orderbook.rs#27-50](#))

```
189 /// Aggregates user orders and liquidity intents for a given sell
token.
190 #[derive(Debug)]
191 pub struct UserLockedAmounts {
192     /// User that submits an order or a liquidity Intent to Turbine
193     pub owner: Address,
194     /// Sell token for orders and intent token for liquidity
provision/withdrawal
195     pub token: Address,
```

Figure D.9: More conventional style of writing rustdoc comments for struct fields
([turbine/src/models.rs#189-195](#))

- Address the code's uses of `todo!()` (shown in figure D.10).

```
state/gateway/trait.rs:53:         todo!()
state/gateway/trait.rs:65:         todo!()
state/gateway/trait.rs:69:         todo!()
state/liquidity_provision/manager.rs:302:         todo!()
```

Figure D.10: Uses of `todo!()` in the code
([turbine/src/state/orderbook.rs#27-50](#))

- Change `SettlementSolution` to `BatchSettlement` in `controller.rs` (figure D.11).

```
17 ///! mid-prices, and cleared (matched) against one another. A final
[`SettlementSolution`] is
18 ///! computed and submitted to the blockchain by the
[`SolutionSubmitter`].
```

Figure D.11: Use of `SettlementSolution` that should be `BatchSettlement` in
`controller.rs` ([turbine/src/controller.rs#17-18](#))

- Remove line 2 of `execution.rs`, shown in figure D.12, as it is outdated and does not prevent any warnings.

```
2    #![allow(dead_code)] // until the builder is used in production code.
```

Figure D.12: Outdated `allow` in `execution.rs` ([turbine/src/execution.rs#2](#))

- Turn comments like those shown in figure D.13 into rustdoc comments for their enclosing method.

```

89    // This method calculates the storage slot for the balance mapping and
    sets the value
90    // directly. It uses a global cache to store known mapping slots for
    efficiency.

```

Figure D.13: Conventional comment that would be better as a rustdoc comment
(*turbine/src/provider_ext.rs#89-90*)

- **Enable Clippy's `arithmetic_side_effects` lint.** When enabled on commit `9644d88`, the lint produces 130 warnings. Figure D.14 gives one example.

```

warning: arithmetic operation that can potentially result in unexpected
side-effects
--> src/price_providers/chainlink.rs:223:29
|
223 |         current_price = current_price * &hop_price;
|                                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
|
= help: for further information visit
https://rust-lang.github.io/rust-clippy/rust-1.91.0/index.html#arithmetic\_side\_effects

```

Figure D.14: An example warning produced by Clippy's `arithmetic_side_effects` lint
(*turbine/src/provider_ext.rs#89-90*)

- **Update the Turbine documentation surrounding the treatment of oracle discrepancies.** The Turbine documentation states that “if an oracle differs by more than 5% from the mean of all oracles,” then “that oracle” is removed. However, The implementation checks this difference against the median, not the mean (see figure D.15). Also, the implementation does not remove the oracle that differs by more than 5%; it removes the most outlying one.

```

let max_distance = self.max_price_deviation * median;

```

Figure D.15: *turbine/src/mid_price_calculator.rs#L169*

- **Delete unused constants.** The constants `TURBINE_SALT` and `UNIVERSAL_ROUTER` in the file `src/constants.rs` are unused. Consider deleting them.
- **Update the documentation of the function `to_f64_lossy`.** The documentation preceding the function `to_f64_lossy` should be updated, as it was deemed OK to be used for non-display purposes and is used as such.

```

/// Approximate conversion of price to f64.
/// This is lossy and should only be used for display purposes.
pub fn to_f64_lossy(&self) -> f64 {
    let numerator = f64::from(self.numerator);
    let denominator = f64::from(self.denominator);

```

```

    numerator / denominator
}

```

Figure D.16: *turbine/src/price.rs#L178-L184*

- **Replace constant literal.** A fee value is set as a constant literal instead of a constant, which makes maintainability harder. Replace that quantity with a constant, or get the fees from the state itself.

```

fee: 30,

```

Figure D.17: *turbine/src/clearing_algorithm/speedex/models.rs#L84*

- **Update inadequate dictionary access.** Accessing a HashMap by key instead of using the get function results in a panic if the key does not exist. Consider replacing all instances of direct access in the file `models.rs` with calls to `get` and perform appropriate error handling. Some examples of direct access are provided in figure D.18.

```

let tick = i32::from(pool.state.state.attributes["tick"].clone());

let sqrt_price = U256::from_be_slice(
    pool.state.state.attributes["sqrt_price_x96"]
        .clone()
        .to_vec()
        .as_slice(),
);

let liquidity = U256::from_be_slice(
    pool.state.state.attributes["liquidity"]
        .clone()
        .to_vec()
        .as_slice(),
);

```

Figure D.18: *turbine/src/clearing_algorithm/speedex/models.rs#L87-L101*

- **Explicitly drop the provider after block subscription polling.** In the function `get_block_feed`, the WebSocket provider is bound to the variable `_provider` (highlighted) and is prefixed with an underscore, which signals to Rust that the value is intentionally unused and may be dropped at any time. This directly violates the warning at line 53 of the same file which states, “save the returned provider in a variable to keep the subscription alive.”

```

loop {
    trace!("Subscribing to blocks");
    let (_provider, mut stream) =

```

```

get_block_subscription(rpc_ws_url.clone()).await;
while let Some(header) = stream.next().await {
    trace!(
        "Got a new block: {}. Delay from block timestamp: {}",
        header.number,
        chrono::Utc::now().timestamp_millis() - header.timestamp as i64 *
1000
    );
    tx.send(header)
        .expect("Failed to broadcast block header");
}
trace!("Block feed channel closed");
}

```

Figure D.19: *turbine/src/block_subscription.rs#L30–L43*

We recommend removing the underscore prefix and explicitly dropping the provider after stream consumption completes. Doing so will make it clear when the stream is no longer needed.

- **In the code in figure D.20, swap the token amounts in addition to the token addresses.** A check later in the same function (`validate_parameters`) will error if the amounts are not swapped. However, swapping the amounts sooner would be more clear.

```

352     if self.add_liquidity.token0 > self.add_liquidity.token1 {
353         swap(&mut self.add_liquidity.token0, &mut
self.add_liquidity.token1);
354     };

```

Figure D.20: Code in `validate_parameters` that swaps token addresses but not amounts (*turbine/src/api/validation.rs#352–354*)

- **Several Solidity loops refer to an array's length (e.g., figure D.21); cache the array's length in a local variable and use the local variable to save gas.**

```

142     for (uint256 i = 0; i < settlement.limitAmounts.length; i++) {
143         initialBalances[i] =
IERC20(settlement.limitAmounts[i].token).balanceOf(address(this));
144     }

```

Figure D.21: Referring to an array's length in a loop (*turbine/contracts/src/settler/TurbineSettler.sol#142–144*)

- **In the comment in figure D.22, change “sell token” to “buy token.”**

```

297     // Send the sell token to the target contract

```

Figure D.22: Comment mentioning “sell token,” but “buy token” is what is sent to the target contract ([turbine/contracts/src/settler/TurbineSettler.sol#297](#))

- **In the comment in figure D.23, swap “hash3” and “hash4.”** “hash4” is too young and “hash3” is expired, not the other way around.

```
1194    // Verify that valid intents (hash1, hash2) and expired intent
(hash4) were removed
1195    // But too young intent (hash3) should NOT be removed
```

*Figure D.23: Test comment referring to several hashes
([turbine/contracts/test/univ4-hook/TurbineHook.t.sol#1194–1195](#))*

E. Recommendations for Addressing Recoverable Errors

Appendix C contains the following known issue or planned improvement:

panic!s, unwraps and expects will be replaced with error handling for all recoverable errors.

In a fork of commit [9644d88](#) with all tests removed, there are 168 occurrences of `panic`, `unwrap`, and `expect`.² Addressing a large fraction of them could be a significant undertaking. This appendix contains some recommendations for doing so.

We would like to emphasize that not panicking should be paramount. As Propeller Heads explained, its Phala Cloud deployment should reboot if it were to terminate abnormally. However, during the review, we observed a case where cargo run could not start because a cache file contained a nonexistent token (see [TOB-TURBINE-1](#)). Addressing such an issue in a live deployment would pose significant challenges.

Panicking in From implementations

Rust's `From` documentation states the following:

Note: This trait must not fail. The `From` trait is intended for perfect conversions. If the conversion can fail or is not perfect, use `TryFrom`.

However, as observed in [TOB-TURBINE-2](#), some `ruint From` implementations panic. We recommend that Turbine not follow the same path. Instead, we recommend using `TryFrom` in Turbine as suggested in the Rust documentation, or fuzzing the Turbine `From` implementations to ensure they cannot panic.

We noticed the following panicking operations in `From` implementations:

- `unwrap` of a `checked_mul` call (figure E.1)
- `unwrap` of a `from_str` call (line 62 in figure E.5)
- `explicit panic` (line 73 in figure E.5)

Replacing Infallible APIs with Fallible APIs

The current Turbine code contains many uses of panicking operations in code that does not return a `Result` type. Rewriting some of these operations will be nontrivial because

² That this, the command `rg -w 'panic|unwrap|expect' src | wc -l` returns 168.

they will require changing the caller's return type. That, in turn, may require changing their caller's return type, and so on. In this subsection, we give one example.

Through fuzzing, we saw the `OrderIntent`'s `From<Order>` implementation panic on line 438 of figure E.1. The most straightforward way to fix this would be to change the function's return type from `Self` to `Result<Self, Self::Error>`. However, such a change would also require changing the trait `From` to `TryFrom`. That, in turn, would require adjusting the function's callers to accommodate the new return type.

```
428     impl From<Order> for OrderIntent {
429         fn from(order: Order) -> Self {
430             Self {
431                 owner: order.owner,
432                 sellToken: order.sell_token,
433                 buyToken: order.buy_token,
434                 sellAmount: order.sell_amount,
435                 minBuyAmount: order
436                     .limit_price
437                     .checked_mul(order.sell_amount)
438                     .unwrap(),
439                 midPriceDelta: order.mid_price_delta,
440                 startTime: order.start_time,
441                 endTime: order.end_time,
442                 partialFill: order.partial_fill,
443                 callData: order.calldata,
444                 callDataTarget: order.calldata_target,
445                 salt: order.salt,
446             }
447         }
448     }
```

Figure E.1: `OrderIntent`'s `From<Order>` implementation ([turbine/src/abi.rs#428-448](#))

Currently, the `From` implementation in figure E.1 is called from one location, the `Order::with_hash` function, shown in figure E.2. Note that this function also does not return a `Result`. Hence, it and its callers will need to change as well.

```
364     pub fn with_hash(mut self) -> Self {
365         let intent = OrderIntent::from(self.clone());
366         let hash = intent.eip712_hash_struct();
367         self.hash = hash;
368         self
369     }
```

Figure E.2: Implementation of `Order::with_hash`. Note that `with_hash` returns `Self`, not `Result<Self, ...>`. Hence, if `OrderIntent`'s `From<Order>` implementation is adjusted to return a `Result`, `with_hash` will need to be adjusted as well.
([turbine/src/models.rs#364-369](#))

Examples of Where Panicking Code Is Currently Used

In this subsection, we give additional examples of panicking code that we encountered through fuzzing. The examples include unwrapping of an Error value (figure E.3), indexing with a nonexistent key (figure E.4), and explicit panics (figure E.5).

```
66     impl RPCGateway {
67         pub fn new(
68             url: &str,
69             turbine_settler_address: Address,
70             lp_hook_address: Address,
71             lp_router_address: Address,
72         ) -> Self {
73             let provider = ProviderBuilder::new()
74                 .connect_http(url.parse().unwrap())
75                 .erased();
76             RPCGateway {
77                 provider: provider.clone(),
78                 erc20_lens: ERC20TokenLens::new(ERC20_LENS_ADDRESS,
provider.clone()),
79                 turbine_settler: ITurbineSettler::new(turbine_settler_address,
provider.clone()),
80                 lp_hook: TurbineHook::new(lp_hook_address, provider.clone()),
81                 lp_router: TurbineLiquidityRouter::new(lp_router_address,
provider.clone()),
82                 permit2: IAllowanceTransferMinimal::new(PERMIT2,
provider.clone()),
83             }
84         }
85     }
```

*Figure E.3: Implementation of `RPCGateway::new`
([turbine/src/blockchain_gateway/rpc.rs#66–85](#))*

```
231     pub(crate) fn from_market_order(
232         sx_clearing: SpeedexMarketOrderClearing,
233         token_metadata: &HashMap<Address, TokenMetadata>,
234     ) -> Self {
235         // Currently, SpeedexMarketOrderClearing.surplus is not used
236         OrderClearing {
237             order_hash: sx_clearing.order_hash,
238             gross_sell_amount: scale_down_floor(
239                 sx_clearing.sold_amount,
240                 token_metadata[&sx_clearing.sell_token].decimals,
241             ),
242             gross_buy_amount: scale_down_ceil(
243                 sx_clearing.bought_amount,
244                 token_metadata[&sx_clearing.buy_token].decimals,
245             ),
246             // Smart orders don't pay fees
```

```

247         buy_token_fees: FeeAbsolute::default(),
248     }
249 }

```

Figure E.4: Implementation of `OrderClearing::from_market_order`
([turbine/src/clearing_algorithm/speedex/models.rs#231-249](#))

```

60     impl From<ComponentWithState> for TychoPool {
61         fn from(component: ComponentWithState) -> Self {
62             let id = Address::from_str(&component.component.id).unwrap();
63             let token0 = bytes_to_address(&component.component.tokens[0]);
64             let token1 = bytes_to_address(&component.component.tokens[1]);
65
66             let protocol = match component
67                 .component
68                 .protocol_system
69                 .as_str()
70             {
71                 "uniswap_v2" => Protocol::UniswapV2,
72                 "uniswap_v3" => Protocol::UniswapV3,
73                 _ => panic!("Unknown protocol: {}",
component.component.protocol_system),
74             };
75
76             Self { id, token0, token1, protocol, state: component }
77         }
78     }

```

Figure E.5: TychoPool's implementation of `From<ComponentWithState>`
([turbine/src/dex_gateway/models.rs#60-78](#))

F. Coverage Analysis

We used `forge coverage` to compute test coverage for the Solidity code³ and `cargo-llvm-cov` to compute test coverage for the Rust code. This appendix summarizes some of what we observed.

Solidity Code

Figure F.1 summarizes the top-level results for the Solidity code. We generally recommend 100% test coverage for Solidity code, as it has the most direct control over user funds.

Directory	Line Coverage ↕			Function Coverage ↕		
	Rate	Total	Hit	Rate	Total	Hit
scripts/univ4-hook/	100.0 %	7	7	100.0 %	1	1
src/settler/	72.9 %	118	86	64.3 %	14	9
src/settler/libraries/	12.5 %	24	3	10.0 %	10	1
src/univ4-hook/	76.9 %	454	349	74.7 %	79	59
src/univ4-hook/libraries/	60.9 %	23	14	100.0 %	4	4
test/	97.3 %	75	73	100.0 %	16	16
test/settler/	42.6 %	54	23	46.2 %	13	6
test/univ4-hook/	60.0 %	40	24	66.7 %	9	6

Figure F.1: Top-level Solidity test coverage results

Note that achieving 100% test coverage will require testing failing (a.k.a. “unhappy”) paths. We give an example in figure F.2. The function `submitRemoveLiquidityIntent` allows users to submit `RemoveLiquidityIntents` to the `TurbineLiquidityRouter` contract. As can be seen in figure F.1, many `revert` statements are uncovered. In other words, none of the Solidity tests exercise those statements. We recommend adding tests and verifying that the following holds:

- When expected to, the `revert` fires.
- The `revert` message returned is the one expected.

³ Specifically, we used the following command: `forge coverage --via-ir --ir-minimum --report=lcov`

```

154 :
155 : // ***** INTENT FUNCTIONS *****
156 :
157 32 : function submitRemoveLiquidityIntent(
158 :     RemoveLiquidityIntent calldata intent,
159 :     AllowanceParams calldata permitParams
160 : ) external returns (RemoveLiquidityIntentHash intentHash) {
161 32 :     if (intent.owner != msg.sender) {
162 0 :         revert InvalidIntentOwner();
163 :     }
164 32 :     if (intent.lpTokenAmount == 0) {
165 0 :         revert InvalidLpTokenAmount();
166 :     }
167 :
168 32 :     (,,, address lpToken,,) = hook.pools(PoolId.unwrap(intent.poolId));
169 :
170 32 :     uint256 balance = IERC20(lpToken).balanceOf(intent.owner);
171 32 :     if (balance < intent.lpTokenAmount) {
172 0 :         revert InsufficientLpTokenBalance(lpToken, intent.lpTokenAmount, balance);
173 :     }
174 :
175 32 :     if (permitParams.permit.details.token != lpToken) {
176 0 :         revert InvalidPermitToken(lpToken, permitParams.permit.details.token);
177 :     }
178 32 :     if (permitParams.permit.details.amount < intent.lpTokenAmount) {
179 0 :         revert InvalidPermitAmount(permitParams.permit.details.amount, intent.lpTokenAmount);
180 :     }
181 32 :     if (permitParams.permit.sigDeadline < permitParams.permit.details.expiration) {
182 0 :         revert PermitSignatureDeadlineTooShort(
183 :             permitParams.permit.sigDeadline, permitParams.permit.details.expiration
184 :         );
185 :     }
186 :
187 32 :     uint256 minExpiration = block.timestamp + MIN_INTENT_EXPIRY;
188 32 :     uint256 maxExpiration = block.timestamp + MAX_INTENT_EXPIRY;
189 32 :     if (permitParams.permit.details.expiration < minExpiration) {
190 0 :         revert PermitExpiryTooShort(permitParams.permit.details.expiration, minExpiration);
191 :     }
192 32 :     if (permitParams.permit.details.expiration > maxExpiration) {
193 1 :         revert PermitExpiryTooLong(permitParams.permit.details.expiration, maxExpiration);
194 :     }
195 31 :     if (permitParams.permit.sigDeadline > maxExpiration) {
196 0 :         revert PermitSignatureDeadlineTooLong(permitParams.permit.sigDeadline, maxExpiration);
197 :     }
198 :
199 31 :     intentHash = _hashRemoveLiquidityIntent(intent);
200 31 :     if (removeLiquidityIntents[intentHash].owner != address(0)) {
201 0 :         revert DuplicateIntent(intentHash);
202 :     }
203 :
204 31 :     removeLiquidityIntents[intentHash] = intent;
205 31 :     removeLiquidityPermits[intentHash] = permitParams;
206 31 :     removeLiquidityCreatedAt[intentHash] = block.timestamp;
207 31 :     intentHashes.push(intentHash);
208 :
209 31 :     emit RemoveLiquidityIntentSubmitted(intent.owner, intent.poolId, intentHash);
210 0 :     return intentHash;
211 : }
212 :

```

Figure F.2: Examples of failing (a.k.a. “unhappy”) paths in `submitRemoveLiquidityIntent`

Rust Code

Figure F.3 gives top-level results for the Rust code. Achieving 100% test coverage of offline code is often impractical. However, the Turbine code runs in a TEE. Hence, Propeller Heads should try to achieve as close to 100% test coverage as possible.

Directory	Line Coverage ↕				Function Coverage ↕		
	Rate		Total	Hit	Rate	Total	Hit
src		84.7 %	6038	5114	42.5 %	1134	482
src/api		95.8 %	3577	3427	67.6 %	479	324
src/blockchain_gateway		85.8 %	254	218	39.5 %	38	15
src/clearing_algorithm		100.0 %	16	16	50.0 %	6	3
src/clearing_algorithm/speedex		96.3 %	1107	1066	62.9 %	105	66
src/dex_gateway		88.2 %	465	410	46.0 %	87	40
src/metrics		94.0 %	116	109	55.0 %	20	11
src/price_providers		86.1 %	1534	1321	58.5 %	241	141
src/solution_submitter		30.8 %	627	193	13.1 %	145	19
src/state		95.8 %	2507	2401	58.1 %	279	162
src/state/gateway		97.3 %	520	506	58.9 %	73	43
src/state/liquidity_provision		93.1 %	1985	1848	54.2 %	203	110

Figure F.3: Top-level Rust test coverage results

The next three figures are not scientific or exhaustive. However, they represent coverage exceptions that we considered notable. Specifically, the uncovered code seems important and thus we feel they should be exercised by tests.

175	:	
176	:	// We don't store both directions on the mid-price hashmap
177	9 :	let mid_price = if let Some(price) = mid_prices.get(pair) {
178	9 :	*price
179	:	} else {
180	0 :	mid_prices[&TokenPair::new(pair.buy_token, pair.sell_token)].inverse()
181	:	};
182	:	

Figure F.4: An excerpt of `src/clearing_algorithm/speedex/runner.rs`'s coverage report. The comment indicates that the `else` case should be executed infrequently. However, the `else` case is not tested.

205	:	
206	10 :	fn normalize_prices(&self, prices: &mut HashMap<Address, U256>) {
207	10 :	if let Some(&min_price) = prices.values().min() {
208	10 :	if min_price < self.base_precision {
209	0 :	for price in prices.values_mut() {
210	0 :	*price = Price::new(self.base_precision, min_price)
211	0 :	.checked_mul(*price)
212	0 :	.unwrap_or(U256::ZERO);
213	0 :	}
214	10 :	}
215	0 :	}
216	10 :	
217	:	
218	:	

Figure F.5: An excerpt of `src/clearing_algorithm/speedex/price_solver.rs`'s coverage report. Note that the loop is never exercised. That is, the tests never normalize prices.

```

121      :
122      :      // Liquidity intents: add the tokens locked via Add or Remove actions.
123      5 :      for action in liquidity_snapshot.actions.iter() {
124      5 :          match action {
125      5 :              LiquidityActionData::AddLiquidity(data) => {
126      5 :                  let owner = data.action.owner;
127      5 :                  let token0 = data.action.token0;
128      5 :                  let token1 = data.action.token1;
129      5 :                  owners_per_token
130      5 :                      .entry(token0)
131      5 :                      .or_default()
132      5 :                      .insert(owner);
133      5 :                  owners_per_token
134      5 :                      .entry(token1)
135      5 :                      .or_default()
136      5 :                      .insert(owner);
137      5 :              }
138      0 :              LiquidityActionData::RemoveLiquidity(data) => {
139      0 :                  let owner = data.action.owner;
140      0 :                  if let Some(pool) = liquidity_snapshot.get_pool(&data.action.pool_id()) {
141      0 :                      let lp_token = pool.metadata.lpToken;
142      0 :                      owners_per_token
143      0 :                          .entry(lp_token)
144      0 :                          .or_default()
145      0 :                          .insert(owner);
146      0 :                  } else {
147      0 :                      warn!(
148      0 :                          "RemoveLiquidity action: pool not found for pool_id {}",
149      0 :                          data.action.pool_id()
150      :                      );
151      :                  }
152      :              }
153      :          }
154      :      }
155      :

```

Figure F.6: An excerpt of `src/validation.rs`'s coverage report. Note that the `LiquidityActionData::RemoveLiquidity` case is not tested at all.

G. Rust Functions That Were Fuzzed

We used `test-fuzz` and `cargo-afl` to fuzz several Turbine functions and methods. This appendix contains a complete list of all functions that we fuzzed.

In total, we fuzzed 196 functions and found panics in 24 of them. To emphasize a point made earlier in the report, finding a panic in an intermediate function does not imply the function could panic from user-provided input.

The table below contains the list of functions that we fuzzed. Specifically, the first column gives the name of the function; the second gives the panic that resulted, if any. If the panic is mentioned in this report, we indicate its location. If a panic was found but is not mentioned in this report, the column simply contains an X.

Function	Panic Found
<code>abi::allowance_params_from</code>	
<code>abi::order_intent_from</code>	Appendix E
<code>abi::permit_batch_from</code>	TOB-TURBINE-3
<code>api::liquidity_provisioning::add_liquidity_action_from</code>	
<code>api::liquidity_provisioning::add_liquidity_intent_eip712_has_h_struct</code>	
<code>api::liquidity_provisioning::add_liquidity_intent_eip712_signing_hash</code>	
<code>api::liquidity_provisioning::add_liquidity_intent_from</code>	
<code>api::liquidity_provisioning::remove_liquidity_action_from</code>	
<code>api::liquidity_provisioning::remove_liquidity_intent_eip712_hash_struct</code>	
<code>api::liquidity_provisioning::remove_liquidity_intent_eip712_signing_hash</code>	
<code>api::liquidity_provisioning::remove_liquidity_intent_from</code>	
<code>api::models::cancel_order_new</code>	
<code>api::models::order_intent_from</code>	

api::models::tee_quote_response_into_response	
api::session_store::extract_user_address	
api::signature::allowance_params_from	
api::signature::order_intent_eip712_hash_struct	
api::signature::permit_batch_eip712_signing_hash	
api::signature::permit_batch_validate_signature	
api::signature::permit_signature_as_bytes	
api::signature::permit_signature_from	
api::signature::permit_signature_recover_address_from_prehash	
api::signature::permit_single_from	Appendix E
api::signature::signed_batch_permit_dto_validate_signature	
api::signature::signed_batch_permit_from	
api::signature::signed_permit_dto_validate_signature	
api::signature::signed_permit_from	
api::validation::validate_adding_to_registered_pool	
api::validation::validate_removing_from_registered_pool	
blockchain_gateway::rpc::rpc_gateway_new	Appendix E
clearing_algorithm::models::order_clearing_executed_buy_amount	
clearing_algorithm::models::order_clearing_executed_sell_amount	
clearing_algorithm::models::order_clearing_new	
clearing_algorithm::speedex::models::order_clearing_from	
clearing_algorithm::speedex::models::order_clearing_from_market_order	Appendix E
clearing_algorithm::speedex::models::pool_clearing_new	

clearing_algorithm::speedex::models::speedex_orderbook_new	
clearing_algorithm::speedex::models::speedex_pool_from	X
clearing_algorithm::speedex::price_solver::token_price_solver_build_graph	
clearing_algorithm::speedex::price_solver::token_price_solver_compute_initial_prices	X
clearing_algorithm::speedex::price_solver::token_price_solver_compute_prices_for_cluster	
clearing_algorithm::speedex::price_solver::token_price_solver_find_base_token	X
clearing_algorithm::speedex::price_solver::token_price_solver_find_token_clusters	
clearing_algorithm::speedex::price_solver::token_price_solver_new	
clearing_algorithm::speedex::price_solver::token_price_solver_normalize_prices	
clearing_algorithm::speedex::runner::speedex_runner_clear_batch	X
clearing_algorithm::speedex::runner::speedex_runner_default	
clearing_algorithm::speedex::utils::scale_down_ceil	
clearing_algorithm::speedex::utils::scale_down_floor	
clearing_algorithm::speedex::utils::scale_up_input	
config::turbine_config_default	
dex_gateway::models::address_to_bytes	
dex_gateway::models::bytes_to_address	
dex_gateway::models::token_metadata_from	X
dex_gateway::models::tycho_pool_from	X
dex_gateway::tycho::pool_states_from	
dex_gateway::tycho::tycho_gateway_new	X

execution::encode_input	
execution::limit_order_execution_from	
execution::smart_order_execution_from	X
metrics::api::create_metrics_api	
metrics::utils::record_duration	
models::exp256	
models::fee_absolute_compute_absolute_fee	
models::fee_absolute_from_fee_hbp	
models::fee_absolute_new	
models::fee_absolute_total_fee_absolute	
models::fee_hbp_new	
models::fee_hbp_total_fee_hbp	
models::increase_price_for_fees	X
models::order_default	
models::order_eq	
models::token_pair_inverse	
models::token_pair_new	
models::token_pair_sorted	
models::update_bytes_at_offset	X
models::user_locked_amounts_aggregate	
models::user_locked_amounts_available_amount	
models::user_locked_amounts_empty	
models::user_locked_amounts_locked_in_liquidity_intents	
models::user_locked_amounts_locked_in_orders	
models::user_locked_amounts_new	

models::user_locked_amounts_set_balance	
models::user_locked_amounts_set_balances_vec	
models::user_locked_amounts_set_permit2_allowance	
models::user_locked_amounts_set_permit2_allowances_vec	X
models::user_locked_amounts_update_total_amount	
price::non_zero_in_non_zero_out	
price::price_apply_delta	
price::price_average	
price::price_checked_mul	
price::price_cmp	
price::price_default	
price::price_div	X
price::price_div_u256	
price::price_eq	
price::price_from	
price::price_from_u512_ratio	X
price::price_identity	
price::price_inverse	
price::price_mul_u256	X
price::price_new	
price::price_partial_cmp	
price::price_reduce	
price::price_to_f64_lossy	
price::price_zero	
price::reduce	

price::reduce_precision	X
price_providers::binance::binance_price_provider_build_request_body	
price_providers::binance::binance_price_provider_new	
price_providers::binance::binance_price_provider_token_to_symbols	
price_providers::market_orders::market_orders_price_provider_default	
price_providers::market_orders::market_orders_price_provider_new	
price_providers::propeller::any_token_pair_from	
price_providers::propeller::calculate_mid_price	X
price_providers::propeller::price_cache_entry_new	
price_providers::propeller::sorted_token_pair_from	
price_providers::propeller::sorted_token_pair_is_sorted	
price_providers::propeller::sorted_token_pair_new	
price_providers::propeller::sorted_token_pair_reversed	
price_providers::pyth::pyth_price_provider_convert_pyth_price	
price_providers::pyth::pyth_price_provider_feed_id_to_token_and_decimals	
price_providers::pyth::pyth_price_provider_new	
price_providers::pyth::pyth_price_provider_token_to_feed	
solution_submitter::r#trait::submission_result_from	
solution_submitter::tenderly::batch_settlement_get_user_token_balances	
solution_submitter::tenderly::tenderly_client_default	
solution_submitter::tenderly::tenderly_client_new	

solution_submitter::tenderly::tenderly_config_default	
solution_submitter::tenderly::tenderly_solution_submitter_new	
solution_submitter::utils::split_remove_liquidity_actions	
state::batch_stats::batch_statistics_calculator_add_batch_stats	
state::batch_stats::batch_statistics_calculator_calculate_cleared_volume_usd	
state::batch_stats::batch_statistics_calculator_calculate_total_batch_fee	
state::batch_stats::batch_statistics_calculator_calculate_total_batch_surplus_usd	
state::batch_stats::batch_statistics_calculator_convert_fees_to_usd	
state::batch_stats::batch_statistics_calculator_default	
state::batch_stats::batch_statistics_calculator_get_batch_stats	
state::batch_stats::batch_statistics_calculator_new	
state::batch_stats::batch_statistics_calculator_process_cleared_batch	
state::liquidity_provision::abi::pool_metadata_id	
state::liquidity_provision::abi::pool_metadata_new	
state::liquidity_provision::abi::pool_metadata_new_default_fee	
state::liquidity_provision::abi::pool_metadata_onchain_id	
state::liquidity_provision::manager::liquidity_manager_default	
state::liquidity_provision::models::add_liquidity_action_pool_id	
state::liquidity_provision::models::calculate_withdrawn_amount	X

nts	
state::liquidity_provision::models::liquidity_action_data_age	
state::liquidity_provision::models::liquidity_action_data_created_at	
state::liquidity_provision::models::liquidity_action_data_intent_hash	
state::liquidity_provision::models::liquidity_action_data_is_expired	
state::liquidity_provision::models::liquidity_action_data_pool_id	
state::liquidity_provision::models::liquidity_action_data_to_ken_pair	
state::liquidity_provision::models::price_checked_add	
state::liquidity_provision::models::remove_liquidity_action_pool_id	
state::liquidity_provision::models::u256_to_naive_datetime	
state::models::liquidity_intent_status_can_transition_to	
state::models::order_execution_new	
state::models::order_state_executed_buy_amount	
state::models::order_state_executed_sell_amount	
state::models::order_state_from	
state::models::order_state_is_filled	
state::models::order_state_last_block_number	
state::models::order_state_overwrite_history	
state::models::order_status_can_transition_to	
state::orderbook::orderbook_add_allowance	
state::orderbook::orderbook_add_order	

state::orderbook::orderbook_add_order_and_allowance	
state::orderbook::orderbook_add_order_settled_events	
state::orderbook::orderbook_add_orders_and_allowances	
state::orderbook::orderbook_cancel	
state::orderbook::orderbook_default	
state::orderbook::orderbook_expire	
state::orderbook::orderbook_fill	
state::orderbook::orderbook_get_all_order_states	
state::orderbook::orderbook_get_allowance	
state::orderbook::orderbook_get_order	
state::orderbook::orderbook_get_state	
state::orderbook::orderbook_has_order	
state::orderbook::orderbook_invalidate	
state::orderbook::orderbook_is_effectively_filled	X
state::orderbook::orderbook_new	
state::orderbook::orderbook_queue_cancellation	
state::orderbook::orderbook_remove_cancelled_orders	
state::orderbook::orderbook_remove_expired_orders	
state::orderbook::orderbook_remove_filled_orders	
state::orderbook::orderbook_remove_orders	X
state::orderbook::orderbook_set_order_executed_sell_amount	
state::orderbook::orderbook_set_order_executed_sell_amounts	
state::orderbook::orderbook_set_status	
state::orderbook::orderbook_snapshot	
state::protocol_state::protocol_state_default	

tee::get_stream	
utils::u256_to_biguint	
validation::remove_orders_and_intents_exceeding_available_amount	X
validation::remove_orders_until_available_amount_enough	

H. Recommendations for Adding a SortedTokenPair Type

The Rust code has a `TokenPair::sorted` method (figure H.1) that sorts its addresses so that the smaller of the two is the first address. We were unable to identify all of the locations where this constraint is relied upon. However, we recommend enforcing the constraint with Rust types, to ensure the constraint is not accidentally overlooked.

```
43 pub fn sorted(self) -> Self {
44     if self.sell_token < self.buy_token {
45         self
46     } else {
47         TokenPair::new(self.buy_token, self.sell_token)
48     }
49 }
```

Figure H.1: Definition of `TokenPair::sorted` ([turbine/src/models.rs#43-49](#))

The type we are recommending would be something like what is shown in figure H.2. Specifically, we recommend sorting the type's addresses in its `new` function, and not allowing the addresses to be changed thereafter. To facilitate interoperability with `TokenPairs`, we recommend implementing `From<TokenPair>` for `SortedTokenPair`.

```
1  #[derive(Clone, Copy, Debug, PartialEq, Eq, Hash, Serialize, Deserialize)]
2  pub struct SortedTokenPair {
3      token_smaller: Address, // Not pub!
4      token_larger: Address,  // Not pub!
5  }
6
7  impl SortedTokenPair {
8      pub fn new(token_a: Address, token_b: Address) -> Self {
9          if token_a < token_b {
10             Self {
11                 token_smaller: token_a,
12                 token_larger: token_b,
13             }
14         } else {
15             Self {
16                 token_smaller: token_b,
17                 token_larger: token_a,
18             }
19         }
20     }
21
22     // Additional functions needed to interact with the `SortedTokenPair`.
23 }
24
```

```
25     impl From<TokenPair> for SortedTokenPair {  
26         fn from(value: TokenPair) -> Self {  
27             Self::new(value.sell_token, value.buy_token)  
28         }  
29     }
```

Figure H.2: Hypothetical definition of the SortedTokenPair type

I. Fix Review Results

When undertaking a fix review, Trail of Bits reviews the fixes implemented for issues identified in the original report. This work involves a review of specific areas of the source code and system configuration, not comprehensive analysis of the system.

From March 12 to March 17, 2026, Trail of Bits reviewed the fixes and mitigations implemented by the Propeller Heads team for the issues identified in this report. We reviewed each fix to determine its effectiveness in resolving the associated issue.

In summary, of the 21 issues described in this report, Propeller Heads has resolved 18 issues and has not resolved the remaining three issues. For additional information, please see the [Detailed Fix Review Results](#) below.

ID	Title	Severity	Status
1	Documentation for running tests needs to be updated	Informational	Resolved
2	serialize_struct argument does not match number of serialize_field calls	Informational	Resolved
3	PermitSingle and PermitBatch's From implementations can panic in U48::from	Informational	Resolved
4	u256_to_naive_datetime can produce values before UNIX epoch	Low	Resolved
5	Invalid bit-length comparison in reduce_precision	Medium	Resolved
6	Recursive price filtering allows oracle threshold bypass	Medium	Unresolved
7	SIWE message version field is not validated	Low	Resolved
8	U256 arithmetic errors silenced with unwrap_or	Informational	Resolved
9	Authentication credentials leak when transmitted over unencrypted HTTP	Medium	Resolved



10	Potential integer overflow in function <code>scale_up_input</code>	High	Resolved
11	ECDSA signature malleability	Low	Resolved
12	Missing validation of user balance vector length	Low	Resolved
13	Missing block number validation in pool state updates	Informational	Resolved
14	Race conditions are present in filesystem operations	Informational	Unresolved
15	Unable to safely change hook address	High	Resolved
16	Unbounded intent array enables denial of service of settlement system	High	Resolved
17	Nonstandard ERC-20 tokens cause swap failures due to unsafe transfer checks	Medium	Resolved
18	Insufficient pre-execution validation enables single-user actions to cause batch settlement failures	Medium	Resolved
19	Remove liquidity operations lack slippage protection, enabling unfavorable token ratios	Medium	Unresolved
20	Off-by-one-second discrepancy in speedbump enforcement between on-chain and off-chain paths	Informational	Resolved
21	Missing minimum buy amount validation allows users to receive less than specified minimum	Informational	Resolved

Detailed Fix Review Results

TOB-TURBINE-1: Documentation for running tests needs to be updated

Resolved in [PR #593](#). Integration tests requiring external services are now gated behind the `test-harness` and `dev` feature flags in `Cargo.toml`, preventing accidental runs without



proper setup. The fix recommended in the finding was also implemented, preventing the modification of production files. Environment variables for on-chain tests (READ_PROVIDER_RPC_URL, TYCHO_AUTH_KEY, TURBINE_SIGNER_PKEY) are documented in DEVELOPMENT.md and the configuration guide.

TOB-TURBINE-2: serialize_struct argument does not match number of serialize_field calls

Resolved in [PR #593](#). The LiquidityManager serialization in `src/state/gateway/file.rs` now correctly specifies `len=2` in the `serialize_struct` call, matching the two `serialize_field` calls that follow (actions and intent_history). This eliminates the undefined behavior that would have resulted from mismatched field counts with non-JSON serializers.

TOB-TURBINE-3: PermitSingle and PermitBatch's From implementations can panic in U48::from

Resolved in [PR #522](#). The PermitSingle implementation has been converted from `From` to `TryFrom` with explicit overflow checks that verify that expiration and nonce values do not exceed `U48::MAX` before conversion, returning a `TypeConversionError` instead of panicking. Additionally, the PermitBatch struct no longer exists in that PR; liquidity intents were migrated to use `SignatureTransfer` instead of `AllowanceTransfer`, thereby addressing the second concern outlined in the finding.

TOB-TURBINE-4: u256_to_naive_datetime can produce values before UNIX epoch

Resolved in [PR #580](#). The function has been refactored to `try_u256_to_naive_datetime`, which returns `Result<NaiveDateTime, TimestampConversionError>`. The implementation uses `TryFrom<U256>` to convert values to `i64`, rejecting values greater than or equal to 2^{63} , then validates that `DateTime::from_timestamp` succeeds.

TOB-TURBINE-5: Invalid bit-length comparison in reduce_precision

Resolved in [PR #426](#) and [PR #464](#). The comparison operator in `reduce_precision` has been changed from `>` to `>=` for validating the bit-length difference against `U256::BITS`. This prevents the edge case where a difference of exactly 256 bits would shift the smaller value to zero.

TOB-TURBINE-6: Recursive price filtering allows oracle threshold bypass

Unresolved. No fix was provided for this finding. Propeller Heads gave the following context:

Price oracles module [is] under redesign.

We recommend reviewing the redesigned price oracle module against this finding once completed.



TOB-TURBINE-7: SIWE message version field is not validated

Resolved in [PR #516](#). The `verify_siwe` function in `src/api/validation.rs` now includes explicit version validation that checks that `message.version` equals `siwe::Version::V1` and returns a `VersionNotSupported` error if the check fails. The validation occurs before other message field checks, correctly rejecting SIWE messages that do not conform to EIP-4361 version 1 requirements.

TOB-TURBINE-8: U256 arithmetic errors silenced with `unwrap_or`

Resolved in [PR #518](#). Some instances have been converted to appropriate patterns: `checked_mul` in `price_solver.rs` now propagates an error via `?`, the `remaining_amount` subtraction in `orderbook.rs` was correctly converted to `saturating_sub`, and the `executed_buy_amount` method in `models.rs` was removed entirely. The silent `U256::ZERO` assignment from an overflowing multiplication in the computation of `surplus_buy_amount` is blocked, as the inner `multiply` now panics via `expect` rather than silently substituting zero. Finally, in the `is_effectively_filled` function, a check verifies that `order.executed_sell_amount` is less than `order.sell_amount`, and a second check verifies that `min_order_fraction` is nonzero.

TOB-TURBINE-9: Authentication credentials leak when transmitted over unencrypted HTTP

Resolved in [PR #522](#). The `TychoGateway::initialize` function in `src/dex_gateway/tycho.rs` now rejects initialization when an authentication key is provided with an HTTP URL, returning an `InsecureAuthentication` error.

TOB-TURBINE-10: Potential integer overflow in function `scale_up_input`

Resolved in [PR #468](#) and [PR #475](#). The `scale_up_input` function now returns `Result<U256, ClearingAlgorithmError>` and uses `checked_mul` to detect multiplication overflows, returning a `ScalingOverflow` error on overflow. The secondary instances in `src/state/liquidity_provision/models.rs` (`calculate_withdrawn_amounts`, `apply_add_liquidity_action`) now use `U512` intermediate calculations to prevent overflow, and then convert back to `U256` via `try_u512_to_u256`, which returns `AmountOverflow` on overflow.

TOB-TURBINE-11: ECDSA signature malleability

Resolved in [PR #468](#). The `recover_address_from_prehash` method in `PermitSignature` now validates that the signature's `s` value does not exceed `SECP256K1N_HALF`, rejecting malleable signatures with `SignatureValidationError::InvalidSignature`.

TOB-TURBINE-12: Missing validation of user balance vector length



Resolved in [PR #522](#). All three affected functions in `src/blockchain_gateway/rpc.rs` (`query_balances`, `query_erc20_allowances`, `query_permit2_permits`) now validate that the multicall response count matches the request count before processing results.

TOB-TURBINE-13: Missing block number validation in pool state updates

Resolved in [PR #584](#). The `update_pool_states` function in `src/dex_gateway/tycho.rs` now validates that incoming block numbers are monotonically increasing. If a block number regresses below `self.last_block`, the update is skipped with a warning logged before any pool state modifications.

TOB-TURBINE-14: Race conditions are present in filesystem operations

Unresolved. Propeller Heads provided the following context:

Outdated. Speedex solver is deprecated. Using a Rust solver library now.

TOB-TURBINE-15: Unable to safely change hook address

Resolved in [PR #481](#). The vulnerability was eliminated through an architectural redesign. `TurbineLiquidityRouter`'s `updateHook` function was replaced with a `setTurbineHook` function that allows the hook to be set only once. The function is called during deployment, after both the router and hook are created. Thus, the `TurbineLiquidityRouter` now holds a reference to the hook, and the hook holds a reference to the router. Furthermore, each `LPToken` stores the hook address at construction. The `hookOnly` modifier in `LPToken` validates that `msg.sender` equals the hook address, preventing the liquidity freeze scenarios described in the finding. See also the changes regarding [PR #531](#) in [appendix K](#).

TOB-TURBINE-16: Unbounded intent array enables denial of service of settlement system

Resolved in [PR #589](#). The `flushExpiredIntents` function was removed along with the `intentHashes` unbounded storage array. An attacker can no longer grow the array to an arbitrary size and cause the `TurbineLiquidityRouter` to iterate over it. Thus, the denial-of-service vector has been eliminated.

TOB-TURBINE-17: Nonstandard ERC-20 tokens cause swap failures due to unsafe transfer checks

Resolved in commit [ffbc792](#) and [PR #664](#). Commit [ffbc792](#) removes `CurrencySettler.sol`. [PR #664](#) adds a test that demonstrates Turbine can swap USDT tokens.

TOB-TURBINE-18: Insufficient pre-execution validation enables single-user actions to cause batch settlement failures

Resolved in [PR #471](#), [PR #462](#), [PR #463](#), [PR #496](#), and [PR #501](#). The `_addLiquidity` and `_removeLiquidity` functions in `TurbineLiquidityRouter.sol` now employ try-catch blocks that return zero amounts on failure instead of reverting the entire batch. The



submitRemoveLiquidityIntent function now pre-validates the LP token balance, permit amounts, deadline bounds, and nonce at submission time.

TOB-TURBINE-19: Remove liquidity operations lack slippage protection, enabling unfavorable token ratios

Unresolved. Propeller Heads provided the following context:

This is by design. Turbine pools ensure that tokens are always swapped at market price or better. So even if “slippage” happens (i.e. reserves ratio changes after the user submits a liquidity removal intent and before the intent gets executed), the user still gets the same value withdrawn. Introducing slippage protection would increase code complexity without clear benefits.

TOB-TURBINE-20: Off-by-one-second discrepancy in speedbump enforcement between on-chain and off-chain paths

Resolved in [PR #541](#). The on-chain `_isTooYoung` function in `TurbineLiquidityRouter.sol` now uses a strictly greater-than comparison (`>`), creating an inclusive boundary where intents created exactly at the speedbump threshold are considered old enough for execution. Both the on-chain and off-chain code enforce the same 12-second speedbump requirement with matching boundary semantics.

TOB-TURBINE-21: Missing minimum buy amount validation allows users to receive less than specified minimum

Resolved in [PR #466](#). The fee calculation was changed to compute fees based on the user's minimum acceptable buy amount rather than the gross buy amount. The change ensures that fees cannot exceed the surplus, preventing the scenario where `totalBuyAmount` falls below the user's minimum. Additionally, Rust-side validation in `src/validation.rs` now checks that `minBuyAmount - buyTokenFees >= limit_amount`. Finally, a new `validate_settlement_balances` function verifies that the settler's token balances remain nonnegative across each batch.

J. Fix Review Status Categories

The following table describes the statuses used to indicate whether an issue has been sufficiently addressed.

Fix Status	
Status	Description
Undetermined	The status of the issue was not determined during this engagement.
Unresolved	The issue persists and has not been resolved.
Partially Resolved	The issue persists but has been partially resolved.
Resolved	The issue has been sufficiently resolved.

K. Review of Known Issues and Planned Improvements PRs

Trail of Bits had time to spare after completing the fix review ([appendix I](#)). We spent the additional time reviewing fixes the Propeller Heads implemented for [Known Issues and Planned Improvements](#). The results of that review appear below.

“We will split off focused roles from `DEFAULT_ADMIN_ROLE`, so that not all permissions are given to a single wallet.”

- **PR #531:** This PR refactors access controls across the three core Turbine contracts.
 - The following fields are now immutable, meaning they cannot be changed after they are set during deployment: `turbineSigner` on `TurbineSettler`, `turbineSettler` on `TurbineLiquidityRouter`, and `router` on `TurbineHook`.
 - `TurbineSettler` gains a dedicated `PAUSER_ROLE`, distinct from the admin role. It also replaces `FEE_WITHDRAWER_ROLE` with a single state variable, `feeWithdrawer`. Finally, it adds a `renounceAdmin` function that allows the caller to renounce their admin role. Note that admins set `turbineLiquidityRouter`. Hence, `renounceAdmin` requires that `turbineLiquidityRouter` is set when it is called.
 - `TurbineLiquidityRouter` was simplified from using OpenZeppelin's `AccessControl` to using Solmate's lighter `Owned` pattern.
 - `TurbineHook` drops `Owned` inheritance entirely, becoming fully trustless.
- **PR #594:** This PR introduces a dedicated `maintainer` address parameter to `TurbineSettler`'s constructor. The `maintainer` parameter receives `PAUSER_ROLE`, a new `UNPAUSER_ROLE`, and the `feeWithdrawer` designation. The PR also adds a “Roles and Permissions” section to `contracts.md` to help make clear the trust relationships between the contracts.

Comments: Consider adding a mechanism to ensure that there is at least one unpauser. Also, consider overriding `TurbineSettler`'s `renounceRole` method and reverting if the caller is the last unpauser. A concern is that all admins could renounce their admin roles, and all unpausers could renounce their unpauser roles. In such a case, the `TurbineSettler` contract could become paused, with no way to unpause it.



“Mid-prices from Propeller Price Provider are not symmetrical (i.e. mid-price for pair A/B is slightly different than inverted price for pair B/A)”

Propeller Heads provided the following information:

Outdated. PropellerPriceProvider got removed.

We verified that PropellerPriceProvider does not appear in the current main branch (ffb8d79).

“Liquidity intents will use **SignatureTransfer instead of **AllowanceTransfer**.”**

- **PR #462:** This PR replaces Permit2 AllowanceTransfer with SignatureTransfer for liquidity intent authorization. AllowanceTransfer sets persistent on-chain allowances that remain active until revoked. By comparison, SignatureTransfer uses single-use nonces that authorize one transfer each and cannot be replayed.
- **PR #463:** This is the Rust companion to PR #462. It updates all liquidity intent permit types and From conversions to use ISignatureTransfer::PermitTransferFrom and PermitBatchTransferFrom in place of IAllowanceTransfer types. Note that Permit2 includes the spender in the signing digest but not in the on-chain struct. To address this, a private permit2_spender_eip712 module defines shadow structs with an added spender field to produce correct EIP-712 typehashes.
- **PR #496:** This PR adds on-chain validation of Permit2 SignatureTransfer nonces for liquidity intents. During validate_locked_amounts, the code queries the nonceBitmap for each intent's nonce, filters out intents whose nonces have already been consumed, and marks them as invalid in the LiquidityManager.
- **PR #501:** This PR splits UserLockedAmounts::total_amount into separate orders_total_amount and liquidity_total_amount fields, reflecting that orders require both balance and permit2 allowance while liquidity intents require only balance.

Comments: PR #501 may have introduced a bug allowing combined overcommitments to not be discovered until settlement time. The old logic correctly enforced that a user's total commitments (orders and liquidity intents) did not exceed their available funds (figures K.1 and K.2). The new logic checks the two quantities independently (figure K.3).

```
if locked.available_amount() >= locked.total_amount {  
    continue;  
}
```



```
}
```

Figure K.1: Old logic for checking a user's available balance
([turbine/src/validation.rs#L297-L299](#))

```
pub fn available_amount(&self) -> U256 {  
    self.available_balance.min(self.available_permit2_allowance)  
}
```

Figure K.2: Excerpt of `models.rs` ([turbine/src/models.rs#L191-L193](#))

```
pub fn enough_available_amount(&self) -> bool {  
    self.available_amount() >= self.orders_total_amount &&  
        self.available_balance >= self.liquidity_total_amount  
}
```

Figure K.3: New logic for checking a user's available balance
([turbine/src/models.rs#L202-L205](#))

A concern is that a user submits an order and an intent that satisfy K.3 independently, but not simultaneously. As a concrete example, a user might do the following:

- Have 100 TOKEN_A balance
- Have 100 TOKEN_A permit2 allowance
- Place an order selling 60 TOKEN_A
- Create a liquidity intent committing 60 TOKEN_A

With the old code (figure K.1), the user's available amount (100) would not have covered the combined amount (120) and validation would have reduced their order. With the new code, validation succeeds and the overcommitment is discovered at settlement time, implying that only one of the two is fulfilled.

Consider adjusting the code in figure K.3 so that overcommitments are caught during validation. Figure K.4 shows a possibility. The left side preserves the new behavior of checking orders against permit2 allowances. The right side is similar to the old behavior of checking that a user's combined intent and order amounts do not exceed their available balance. Note that the proposed change preserves the PR's intent of checking liquidity intents against balance only, and not allowance.

```
pub fn enough_available_amount(&self) -> bool {  
    self.available_amount() >= self.orders_total_amount &&  
        self.available_balance  
            >= self.liquidity_total_amount + self.orders_total_amount
```

```
}
```

Figure K.4: Proposed logic for catching overcommitments during validation

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at <https://github.com/trailofbits/publications>, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow [@trailofbits on X](#) or [LinkedIn](#) and explore our public repositories at <https://github.com/trailofbits>. To engage us directly, visit our "Contact" page at <https://www.trailofbits.com/contact> or email us at info@trailofbits.com.

Trail of Bits, Inc.

228 Park Ave S #80688

New York, NY 10003

<https://www.trailofbits.com>

info@trailofbits.com



Notices and Remarks

Copyright and Distribution

© 2026 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

Trail of Bits considers this report public information; it is licensed to Propeller Heads under the terms of the project statement of work and has been made public at Propeller Heads' request. Material within this report may not be reproduced or distributed in part or in whole without Trail of Bits' express written permission.

The sole canonical source for Trail of Bits publications is the [Trail of Bits Publications page](#). Reports accessed through sources other than that page may have been modified and should not be considered authentic.

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.

